
ЧАСТЬ IV

ДОПОЛНИТЕЛЬНЫЕ ТЕМЫ

В этой части...

Глава 17. Специализированные средства библиотек

Глава 18. Инструменты для крупномасштабных программ

Глава 19. Специализированные инструменты и технологии

Часть IV посвящена дополнительным средствам, которые весьма полезны в некоторых случаях, но нужны не каждому разработчику на языке C++. Эти средства делятся на две группы: те, которые используются для решения крупномасштабных проблем, и те, которые применяют скорее для специфических целей, а не общих. Средства для специфических задач, предоставляемые языком, рассматриваются в главе 19, а таковые, предоставленные библиотекой, — в главе 17.

В главе 17 рассматриваются четыре библиотечных средства специального назначения: класс `bitset` (набора битов) и три новых библиотечных средства: кортежи, регулярные выражения и случайные числа. Затронуты также будут и некоторые из менее общеизвестных частей библиотеки ввода и вывода.

Глава 18 посвящена обработке исключений, пространствам имен и множественному наследованию. Эти средства могут быть весьма полезны в контексте крупномасштабных программ.

Даже достаточно простые программы, которые могут быть написаны одним разработчиком, способны извлечь пользу из обработки исключений, основы которой были представлены в главе 5. Однако необходимость справляться с непредвиденными ошибками во время выполнения программы не менее важна, чем решение проблем в больших группах разработчиков. В главе 18 представлен обзор некоторых дополнительных средств обработки исключений. Здесь также более подробно рассматриваются способы обработки исключений, их смысл при размещении ресурсов в памяти и их удалении. Кроме того, в этой главе описаны способы создания и применения собственных классов исключений, рассматриваются также усовершенствования из нового стандарта, включая определение того, что некая функция не будет передавать исключения.

В крупномасштабных приложениях зачастую используют код от нескольких независимых производителей. Комбинирование нескольких библиотек от независимых разработчиков было бы необычайно трудной или вообще неразрешимой задачей, если бы все использованные в них имена располагались в одном пространстве имен. В библиотеках от независимых разработчиков почти неизбежно использовались бы совпадающие имена. В результате имя, определенное в одной библиотеке, вступило бы в конфликт с таким же именем из другой библиотеки. Чтобы избежать конфликтов имен, их следует определять в *пространстве имен* (namespace).

Каждый раз, когда в этой книге использовалось имя из стандартной библиотеки, происходило обращение к пространству имен `std`. В главе 18 продемонстрировано, как можно определять собственные пространства имен.

Глава 18 завершается очень важным, но нечасто используемым средством языка: множественным наследованием. Множественное наследование наиболее полезно в сложных иерархиях наследования.

Глава 19 посвящена ряду специализированных подходов и инструментальных средств решения ряда специфических проблем. В этой главе рассматриваются такие средства, как дополнительные возможности по распределению памяти; поддержка языком C++ идентификации типов времени выполнения (RTTI), позволяющей определять фактический тип выражения во время выполнения; а также способы определения и использования указателей на члены класса. Указатели на члены классов отличаются от указателей на обычные данные или функции. Обычные указатели различаются только на основании типа объекта или функции. Указатели на члены класса должны также отражать класс, которому принадлежит член. Затем рассматриваются три дополнительных составных типа: объединения, вложенные и локальные классы. Глава завершается кратким обзором средств, применение которых делает код переносимым. Сюда относится спецификатор `volatile`, битовые поля и директивы компоновки.

ГЛАВА 17

СПЕЦИАЛИЗИРОВАННЫЕ СРЕДСТВА БИБЛИОТЕК

В этой главе...

17.1. Тип <code>tuple</code>	899
17.2. Тип <code>bitset</code>	906
17.3. Регулярные выражения	912
17.4. Случайные числа	933
17.5. Еще о библиотеке ввода и вывода	943
Резюме	961
Термины	962

Последний стандарт существенно увеличил размер и область видимости библиотеки. Действительно, посвященная библиотеке часть стандарта более чем удвоилась по сравнению с прежним выпуском стандарта и составила почти две трети текста нового стандарта. В результате подробное рассмотрение каждого класса библиотеки C++ стало невозможным в данном издании. Однако четыре специализированных библиотечных средства являются достаточно общими, чтобы рассмотреть их в данной книге: это кортежи, наборы битов, генераторы случайных чисел и регулярные выражения. Кроме того, будут рассмотрены также некоторые дополнительные специальные средства библиотеки ввода и вывода.

17.1. Тип `tuple`

**C++
11** Шаблон `tuple` (кортеж) подобен шаблону `pair` (пара) (см. раздел 11.2.3, стр. 545). У каждого экземпляра шаблона `pair` могут быть члены разных типов, но их всегда только два. Члены экземпляров шаблона `tuple` также могут иметь разные типы, но количество их может быть любым. Каждый конкретный экземпляр шаблона `tuple` имеет фиксированное количество членов, но другой экземпляр типа может отличаться количеством членов.

Тип `tuple` особенно полезен, когда необходимо объединить некие данные в единый объект, но нет желания определять структуру для их хранения. Список операций, поддерживаемых типом `tuple`, приведен в табл. 17.1. Тип

`tuple`, наряду с сопутствующими ему типами и функциями, определен в заголовке `tuple`.

Таблица 17.1. Операции с кортежами

<code>tuple<T1, T2, ..., Tn> t;</code>	<code>t</code> — кортеж с количеством и типами членов, заданными списком <code>T1...Tn</code> . Члены инициализируются по умолчанию (см. раздел 3.3.1, стр. 143)
<code>tuple<T1, T2, ..., Tn> t(v1, v2, ..., vn);</code>	<code>t</code> — кортеж с типами <code>T1...Tn</code> , каждый член которого инициализируется соответствующим инициализатором <code>vi</code> . Этот конструктор является явным (см. раздел 7.5.4, стр. 383)
<code>make_tuple(v1, v2, ..., vn)</code>	Возвращает кортеж, инициализированный данными инициализаторов. Тип кортежа выводится из типов инициализаторов
<code>t1 == t2</code> <code>t1 != t2</code>	Два кортежа равны, если у них совпадает количество членов и каждая пара членов равна. Для сравнения используется собственный оператор <code>==</code> каждого члена. Как только найдены неравные члены, последующие не проверяются
<code>t1 опсравн t2</code>	Операторы сравнения кортежей используют алфавитный порядок (см. раздел 9.2.7, стр. 438). У кортежей должно быть одинаковое количество членов. Члены кортежа <code>t1</code> сравниваются с соответствующими членами кортежа <code>t2</code> при помощи оператора <code><</code>
<code>get<i>(t)</code>	Возвращает ссылку <code>i</code> -ю переменную-член кортежа <code>t</code> ; если <code>t</code> — это <code>l</code> -значение, то результат — ссылка на <code>l</code> -значение; в противном случае — ссылка на <code>r</code> -значение. Все члены кортежа являются открытыми (<code>public</code>)
<code>tuple_size<типКортежа>::value</code>	Шаблон класса, экземпляр которого может быть создан по типу кортежа и имеет <code>public constexpr static</code> переменную-член <code>value</code> типа <code>size_t</code> , содержащую количество членов в указанном типе кортежа
<code>tuple_element<i, типКортежа>::type</code>	Шаблон класса, экземпляр которого может быть создан по целочисленной константе и типу кортежа, имеющий открытый член <code>type</code> , являющийся типом указанного члена в кортеже указанного типа



Тип tuple можно считать структурой данных на “скорую руку”.

17.1.1. Определение и инициализация кортежей

При определении кортежа следует указать типы каждого из его членов:

```
tuple<size_t, size_t, size_t> threeD; // все три члена установлены в 0
tuple<string, vector<double>, int, list<int>>
    someVal("constants", {3.14, 2.718}, 42, {0,1,2,3,4,5});
```

При создании объекта кортежа можно использовать либо стандартный конструктор кортежа, инициализирующий каждый член по умолчанию (см. раздел 3.3.1, стр. 143), либо предоставить инициализатор для каждого члена, как при инициализации кортежа `someVal`. Этот конструктор кортежа является явным (см. раздел 7.5.4, стр. 383), поэтому следует использовать прямой синтаксис инициализации:

```
tuple<size_t, size_t, size_t> threeD = {1,2,3}; // ошибка
tuple<size_t, size_t, size_t> threeD{1,2,3};    // ok
```

В качестве альтернативы, подобно функции `make_pair()` (см. раздел 11.2.3, стр. 547), можно использовать библиотечную функцию `make_tuple()`, создающую объект кортежа:

```
// кортеж, представляющий транзакцию приложения книжного магазина: ISBN,
// количество, цена книги
auto item = make_tuple("0-999-78345-X", 3, 20.00);
```

Подобно функции `make_pair()`, функция `make_tuple()` использует типы, предоставляемые в качестве инициализаторов, для вывода типа кортежа. В данном случае кортеж `item` имеет тип `tuple<const char*, int, double>`.

Доступ к членам кортежа

В типе `pair` всегда есть два члена, что позволяет библиотеке присвоить им имена `first` (первый) и `second` (второй). Для типа `tuple` такое соглашение об именовании невозможно, поскольку у него нет ограничений на количество членов. В результате члены остаются безымянными. Вместо имен для обращения к членам кортежа используется библиотечный шаблон функции `get`. Чтобы использовать шаблон `get`, следует определить явный аргумент шаблона (см. раздел 16.2.2, стр. 857), задающий позицию члена, доступ к которому предстоит получить. Функция `get()` получает объект кортежа и возвращает ссылку на его заданный член:

```
auto book = get<0>(item);           // возвращает первый член item
auto cnt = get<1>(item);           // возвращает второй член item
auto price = get<2>(item)/cnt;     // возвращает последний член item
get<2>(item) *= 0.8;               // применяет 20%-ную скидку
```

Значение в скобках должно быть целочисленным константным выражением (см. разделе 2.4.4, стр. 103). Как обычно, счет начинается с 0, а значит, первым членом будет `get<0>`.

Если подробности типов в кортеже неизвестны, для выяснения количества и типов его членов можно использовать два вспомогательных шаблона класса:

```
typedef decltype(item) trans;           // trans - тип кортежа item
// возвращает количество членов в объекте типа trans
size_t sz = tuple_size<trans>::value;   // возвращает 3
// cnt имеет тот же тип, что и второй член item
tuple_element<1, trans>::type cnt = get<1>(item); // cnt - это int
```

Для использования шаблонов `tuple_size` и `tuple_element` необходимо знать тип объекта кортежа. Как обычно, проще всего определить тип объекта при помощи спецификатора `decltype` (см. раздел 2.5.3, стр. 110). Здесь спецификатор `decltype` используется для определения псевдонима для типа кортежа `item`, который и используется при создании экземпляров обоих шаблонов.

Шаблон `tuple_size` обладает открытой статической переменной-членом `value`, содержащей количество членов в указанном кортеже. Шаблон `tuple_element` получает индекс, а также тип кортежа. Он обладает открытым типом-членом `type`, содержащим тип указанного члена кортежа заданного типа. Подобно функции `get()`, шаблон `tuple_element` ведет отсчет индексов начиная с нуля.

Операторы сравнения и равенства

Операторы сравнения и равенства кортежей ведут себя подобно соответствующим операторам контейнеров (см. раздел 9.2.7, стр. 438). Эти операторы выполняются для членов двух кортежей, слева и справа. Сравнить два кортежа можно только при совпадении количества их членов. Кроме того, чтобы использовать операторы равенства или неравенства, должно быть допустимо сравнение каждой пары членов при помощи оператора `==`; а для использования операторов сравнения допустимым должно быть использование оператора `<`. Например:

```
tuple<string, string> duo("1", "2");
tuple<size_t, size_t> twoD(1, 2);
bool b = (duo == twoD); // ошибка: нельзя сравнить size_t и string
tuple<size_t, size_t, size_t> threeD(1, 2, 3);
b = (twoD < threeD);    // ошибка: разное количество членов
tuple<size_t, size_t> origin(0, 0);
b = (origin < twoD);    // ok: b - это true
```



Поскольку кортеж определяет операторы `<` и `==`, последовательности кортежей можно передавать алгоритмам, а также использовать кортеж как тип ключа в упорядоченном контейнере.

Упражнения раздела 17.1.1

Упражнение 17.1. Определите кортеж, содержащий три члена типа `int`, и инициализируйте их значениями 10, 20 и 30.

Упражнение 17.2. Определите кортеж, содержащий строку, вектор строки и пару из строки и целого числа (типы `string`, `vector<string>` и `pair<string, int>`).

Упражнение 17.3. Перепишите программы `TextQuery` из раздела 12.3 (стр. 617) так, чтобы использовать кортеж вместо класса `QueryResult`. Объясните, что на ваш взгляд лучше и почему.

17.1.2. Использование кортежей для возвращения нескольких значений

Обычно кортеж используют для возвращения из функции нескольких значений. Например, рассматриваемый книжный магазин мог бы быть одним из нескольких магазинов в сети. У каждого магазина был бы транзакционный файл, содержащий данные по каждой проданной книге. В этом случае могло бы понадобиться просмотреть все продажи данной книги по всем магазинам.

Предположим, для каждого магазина имеется файл транзакций. Каждый из этих транзакционных файлов в магазине будет содержать все транзакции для каждой группы книг. Предположим также, что некая другая функция читает эти транзакционные файлы, создает вектор `vector<Sales_data>` для каждого магазина и помещает эти векторы в вектор векторов:

```
// каждый элемент в файле содержит транзакции для определенного магазина
vector<vector<Sales_data>> files;
```

Давайте напишем функцию, которая будет просматривать файлы в поисках магазина, продавшего заданную книгу. Для каждого магазина, у которого есть соответствующая транзакция, необходимо создать кортеж для содержания индекса этого магазина и двух итераторов. Индекс будет позицией соответствующего магазина в файлах, а итераторы отметят первую и следующую после последней записи по заданной книге в векторе `vector<Sales_data>` этого магазина.

Функция, возвращающая кортеж

Для начала напишем функции поиска заданной книги. Аргументами этой функции будет только что описанный вектор векторов и строка, представляющая ISBN книги. Функция будет возвращать вектор кортежей с записями по каждому магазину, где была продана по крайней мере одна заданная книга:

```

// matches имеет три члена: индекс магазина и итераторы в его векторе
typedef tuple<vector<Sales_data>::size_type,
            vector<Sales_data>::const_iterator,
            vector<Sales_data>::const_iterator> matches;
// files хранит транзакции по каждому магазину
// findBook() возвращает вектор с записями для каждого магазина,
// продавшего данную книгу
vector<matches>
findBook(const vector<vector<Sales_data>> &files,
        const string &book)
{
    vector<matches> ret; // изначально пуст
    // для каждого магазина найти диапазон, соответствующий книге
    // (если он есть)
    for (auto it = files.cbegin(); it != files.cend(); ++it) {
        // найти диапазон Sales_data с тем же ISBN
        auto found = equal_range(it->cbegin(), it->cend(),
                                book, compareIsbn);
        if (found.first != found.second) // у этого магазина есть продажи
            // запомнить индекс этого магазина и диапазона соответствий
            ret.push_back(make_tuple(it - files.cbegin(),
                                    found.first, found.second));
    }
    return ret; // пуст, если соответствий не найдено
}

```

Цикл `for` перебирает элементы вектора `files`, которые сами являются векторами. В цикле `for` происходит вызов библиотечного алгоритма `equal_range()`, работающего как одноименная функция-член ассоциативного контейнера (см. раздел 11.3.5, стр. 561). Первые два аргумента функции `equal_range()` являются итераторами, обозначающими исходную последовательность (см. раздел 10.1, стр. 482). Третий аргумент — значение. По умолчанию для сравнения элементов функция `equal_range()` использует оператор `<`. Поскольку тип `Sales_data` не имеет оператора `<`, передаем указатель на функцию `compareIsbn()` (см. раздел 11.2.2, стр. 543).

Алгоритм `equal_range()` возвращает пару итераторов, обозначающих диапазон элементов. Если книга не будет найдена, то итераторы окажутся равны, означая, что диапазон пуст. В противном случае первый член возвращенной пары обозначит первую соответствующую транзакцию, а второй — следующую после последней.

Использование возвращенного функцией кортежа

После создания вектора магазинов с соответствующей транзакцией эти транзакции необходимо обработать. В данной программе следует сообщить результаты общего объема продаж для каждого магазина, у которого была такая продажа:

```

void reportResults(istream &in, ostream &os,
                 const vector<vector<Sales_data>> &files)

```

```

{
    string s; // искомая книга
    while (in >> s) {
        auto trans = findBook(files, s); // магазин, продавший эту книгу
        if (trans.empty()) {
            cout << s << " not found in any stores" << endl;
            continue; // получить следующую книгу для поиска
        }
        for (const auto &store : trans) // для каждого магазина с
            // продаж
            // get<n> возвращает указанный элемент кортежа в store
            os << "store " << get<0>(store) << " sales: "
                << accumulate(get<1>(store), get<2>(store),
                               Sales_data(s))
                << endl;
    }
}

```

Цикл `while` последовательно читает поток `istream` по имени `in`, чтобы запустить обработку следующей книги. Вызов функции `findBook()` позволяет выяснить, присутствует ли строка `s`, и присваивает результаты вектору `trans`. Чтобы упростить написание типа `trans`, являющегося вектором кортежей, используем ключевое слово `auto`.

Если вектор `trans` пуст, значит, по книге `s` никаких продаж не было. В таком случае выводится сообщение и происходит возврат к циклу `while`, чтобы обработать следующую книгу.

Цикл `for` свяжет ссылку `store` с каждым элементом вектора `trans`. Поскольку изменять элементы вектора `trans` не нужно, объявим ссылку `store` ссылкой на константу. Для вывода результатов используем `get`: `get<0>` — индекс соответствующего магазина; `get<1>` — итератор на первую транзакцию; `get<2>` — на следующую после последней.

Поскольку класс `Sales_data` определяет оператор суммы (см. раздел 14.3, стр. 710), для суммирования транзакций можно использовать библиотечный алгоритм `accumulate()` (см. раздел 10.2.1, стр. 485). Как отправную точку суммирования используем объект класса `Sales_data`, инициализированный конструктором `Sales_data()`, получающим строку (см. раздел 7.1.4, стр. 345). Этот конструктор инициализирует переменную-член `bookNo` переданной строкой, а переменные-члены `units_sold` и `_revenue` — нулем.

Упражнения раздела 17.1.2

Упражнение 17.4. Напишите и проверьте собственную версию функции `findBook()`.

Упражнение 17.5. Перепишите функцию `findBook()` так, чтобы она возвращала пару, содержащую индекс и пару итераторов.

Упражнение 17.6. Перепишите функцию `findBook()` так, чтобы она не использовала кортеж или пару.

Упражнение 17.7. Объясните, какую версию функции `findBook()` вы предпочитаете и почему.

Упражнение 17.8. Что будет, если в качестве третьего параметра алгоритма `accumulate()` в последнем примере кода этого раздела передать объект класса `Sales_data`?

17.2. Тип `bitset`

В разделе 4.8 (стр. 210) приводились встроенные операторы, рассматривающие целочисленный операнд как коллекцию битов. Для облегчения использования битовых операций и обеспечения возможности работы с коллекциями битов, размер которых больше самого длинного целочисленного типа, стандартная библиотека определяет класс `bitset` (набор битов). Класс `bitset` определен в заголовке `bitset`.

17.2.1. Определение и инициализация наборов битов

Список конструкторов типа `bitset` приведен в табл. 17.2. Тип `bitset` — это шаблон класса, который, подобно классу `array`, имеет фиксированный размер (см. раздел 3.3.6, стр. 432). При определении набора битов следует указать в угловых скобках количество битов, которые он будет содержать:

```
bitset<32> bitvec(1U); // 32 бита; младший бит 1, остальные биты 0
```

Размер должен быть указан константным выражением (см. раздел 2.4.4, стр. 103). Этот оператор определяет набор битов `bitvec`, содержащий 32 бита. Подобно элементам вектора, биты в наборе битов не имеют имен. Доступ к ним осуществляется по позиции. Нумерация битов начинается с 0. Таким образом, биты набора `bitvec` пронумерованы от 0 до 31. Биты, расположенные ближе к началу (к 0), называются *младшими битами* (low-order), а ближе к концу (к 31) — *старшими битами* (high-order).

Таблица 17.2. Способы инициализации набора битов

<code>bitset<n> b;</code>	Набор <code>b</code> содержит <code>n</code> битов, каждый из которых содержит значение 0. Это конструктор <code>constexpr</code> (см. раздел 7.5.6, стр. 385)
<code>bitset<n> b(u);</code>	Набор <code>b</code> содержит копию <code>n</code> младших битов значения <code>u</code> типа <code>unsigned long long</code> . Если значение <code>n</code> больше размера типа <code>unsigned long long</code> , остальные старшие биты устанавливаются на нуль. Это конструктор <code>constexpr</code> (см. раздел 7.5.6, стр. 385)

Окончание табл. 17.2

<code>bitset<n> b(s, pos, m, zero, one);</code>	Набор <code>b</code> содержит копию <code>m</code> символов из строки <code>s</code> , начиная с позиции <code>pos</code> . Строка <code>s</code> может содержать только символы для нулей и единиц; если строка <code>s</code> содержит любой другой символ, передается исключение <code>invalid_argument</code> . Символы хранятся в наборе <code>b</code> как нули и единицы соответственно. По умолчанию параметр <code>pos</code> имеет значение 0, параметр <code>m</code> — <code>string::npos</code> , <code>zero</code> — <code>'0'</code> и <code>one</code> — <code>'1'</code>
<code>bitset<n> b(cp, pos, m, zero, one);</code>	Подобен предыдущему конструктору, но копируется символьный массив, на который указывает <code>cp</code> . Если значение <code>m</code> не предоставлено, <code>cp</code> должен указывать на строку в стиле C. Если <code>m</code> предоставлено, то начиная с позиции <code>cp</code> в массиве должно быть по крайней мере <code>m</code> символов, соответствующих нулям или единицам

Конструкторы, получающие строку или символьный указатель, являются явными (см. раздел 7.5.4, стр. 383). В новом стандарте была добавлена возможность определять альтернативные символы для 0 и 1.

Инициализация набора битов беззнаковым значением

При использовании для инициализации набора битов целочисленного значения оно преобразуется в тип `unsigned long long` и рассматривается как битовая схема. Биты в наборе битов являются копией этой схемы. Если размер набора битов превосходит количество битов в типе `unsigned long long`, то остальные старшие биты устанавливаются в нуль. Если размер набора битов меньше количества битов, то будут использованы только младшие биты предоставленного значения, а старшие биты вне размера объекта набора битов отбрасываются:

```
// bitvec1 меньше инициализатора; старшие биты инициализатора
// отбрасываются
bitset<13> bitvec1(0xbeef); // биты 111101110111
// bitvec2 больше инициализатора; старшие биты bitvec2 устанавливаются в
// нуль
bitset<20> bitvec2(0xbeef); // биты 000010111101110111
// на машинах с 64-битовым long long, OULL - это 64 бита из 0,
// а ~OULL - 64 единицы
bitset<128> bitvec3(~OULL); // биты 0...63 - единицы; 63...127 - нули
```

Инициализация набора битов из строки

Набор битов можно инициализировать из строки или указателя на элемент в символьном массиве. В любом случае символы непосредственно представляют битовую схему. Как обычно, при использовании строки для представления числа символы с самыми низкими индексами в строке соответствуют старшим битам, и наоборот:

```
bitset<32> bitvec4("1100"); // биты 2 и 3 - единицы, остальные - 0
```

Если строка содержит меньше символов, чем битов в наборе, старшие биты устанавливаются в нуль.



Соглашения по индексации строк и наборов битов прямо противоположны: символ строки с самым высоким индексом (крайний правый символ) используется для инициализации младшего бита в наборе битов (бит с индексом 0). При инициализации набора битов из строки следует помнить об этом различии.

Необязательно использовать всю строку в качестве исходного значения для набора битов, вполне можно использовать часть строки:

```
string str("1111111000000011001101");
bitset<32> bitvec5(str, 5, 4); // четыре бита, начиная с str[5] - 1100
bitset<32> bitvec6(str, str.size()-4); // использует четыре последних
// символа
```

Здесь набор битов `bitvec5` инициализируется подстрокой `str`, начиная с символа `str[5]`, и четырьмя символами далее. Как обычно, крайний справа символ подстроки представляет бит самого низкого порядка. Таким образом, набор `bitvec5` инициализируется битами с позиции 3 до 0 и получает значение 1100, а остальные биты — 0. Инициализатор набора битов `bitvec6` передает строку и отправную точку, поэтому он инициализируется символами строки `str`, начиная с четвертого и до конца строки `str`. Остаток битов набора `bitvec6` инициализируется нулями. Эти инициализации можно представить так:



Упражнения раздела 17.2.1

Упражнение 17.9. Объясните битовую схему, которую содержит каждый из следующих объектов `bitset`:

- (a) `bitset<64> bitvec(32);`
- (b) `bitset<32> bv(1010101);`
- (c) `string bstr; cin >> bstr; bitset<8>bv(bstr);`

17.2.2. Операции с наборами битов

Операции с наборами битов (табл. 17.3) определяют различные способы проверки и установки одного или нескольких битов. Класс `bitset` поддерживает также побитовые операторы, которые рассматривались в разделе 4.8 (стр. 210). Применительно к объектам `bitset` эти операторы имеют тот же смысл, что и таковые встроенные операторы для типа `unsigned`.

Таблица 17.3. Операции с наборами битов

<code>b.any()</code>	Установлен ли в наборе <code>b</code> хоть какой-нибудь бит?
<code>b.all()</code>	Все ли биты набора <code>b</code> установлены?
<code>b.none()</code>	Нет ли в наборе <code>b</code> установленных битов?
<code>b.count()</code>	Количество установленных битов в наборе <code>b</code>
<code>b.size()</code>	Функция <code>constexpr</code> (см. раздел 2.4.4, стр. 103), возвращающая количество битов набора <code>b</code>
<code>b.test(pos)</code>	Возвращает значение <code>true</code> , если бит в позиции <code>pos</code> установлен, и значение <code>false</code> в противном случае
<code>b.set(pos, v)</code> <code>b.set()</code>	Устанавливает для бита в позиции <code>pos</code> логическое значение <code>v</code> . По умолчанию <code>v</code> имеет значение <code>true</code> . Без аргументов устанавливает все биты набора <code>b</code>
<code>b.reset(pos)</code> <code>b.reset()</code>	Сбрасывает бит в позиции <code>pos</code> или все биты набора <code>b</code>
<code>b.flip(pos)</code> <code>b.flip()</code>	Изменяет состояние бита в позиции <code>pos</code> или все биты набора <code>b</code>
<code>b[pos]</code>	Предоставляет доступ к биту набора <code>b</code> в позиции <code>pos</code> ; если набор <code>b</code> константен и бит установлен, то <code>b[pos]</code> возвращает логическое значение <code>true</code> , а в противном случае — значение <code>false</code>
<code>b.to_ulong()</code> <code>b.to_ullong()</code>	Возвращает значение типа <code>unsigned long</code> или типа <code>unsigned long long</code> с теми же битами, что и в наборе <code>b</code> . Если битовая схема в наборе <code>b</code> не соответствует указанному типу результата, передается исключение <code>overflow_error</code>

Окончание табл. 17.3

<code>b.to_string(zero, one)</code>	Возвращает строку, представляющую битовую схему набора <code>b</code> . Параметры <code>zero</code> и <code>one</code> имеют по умолчанию значения <code>'0'</code> и <code>'1'</code> . Они используются для представления битов 0 и 1 в наборе <code>b</code>
<code>os << b</code>	Выводит в поток <code>os</code> биты набора <code>b</code> как символы <code>'0'</code> и <code>'1'</code>
<code>is >> b</code>	Читает символы из потока <code>is</code> в набор <code>b</code> . Чтение прекращается, когда следующий символ отличается от 1 или 0 либо когда прочитано <code>b.size()</code> битов

Некоторые из функций, `count()`, `size()`, `all()`, `any()` и `none()`, не принимают аргументов и возвращают информацию о состоянии всего набора битов. Другие, `set()`, `reset()` и `flip()`, изменяют состояние набора битов. Функции-члены, изменяющие набор битов, допускают перегрузку. В любом случае версия функции без аргументов применяет соответствующую операцию ко всему набору, а версии функций, получающих позицию, применяют операцию к заданному биту:

```
bitset<32> bitvec(1U); // 32 бита; младший бит 1, остальные биты - 0
bool is_set = bitvec.any(); // true, установлен один бит
bool is_not_set = bitvec.none(); // false, установлен один бит
bool all_set = bitvec.all(); // false, только один бит установлен
size_t onBits = bitvec.count(); // возвращает 1
size_t sz = bitvec.size(); // возвращает 32
bitvec.flip(); // инвертирует значения всех битов в bitvec
bitvec.reset(); // сбрасывает все биты в 0
bitvec.set(); // устанавливает все биты в 1
```

**C++
11** Функция `any()` возвращает значение `true`, если один или несколько битов объекта класса `bitset` установлены, т.е. равны 1. Функция `none()`, наоборот, возвращает значение `true`, если все биты содержат нуль. Новый стандарт ввел функцию `all()`, возвращающую значение `true`, если все биты установлены. Функции `count()` и `size()` возвращают значение типа `size_t` (см. раздел 3.5.2, стр. 165), равное количеству установленных битов, или общее количество битов в объекте соответственно. Функция `size()` — `constexpr`, а значит, она применима там, где требуется константное выражение (см. раздел 2.4.4, стр. 103).

Функции `flip()`, `set()`, `reset()` и `test()` позволяют читать и записывать биты в заданную позицию:

```
bitvec.flip(0); // инвертирует значение первого бита
bitvec.set(bitvec.size() - 1); // устанавливает последний бит
bitvec.set(0, 0); // сбрасывает первый бит
bitvec.reset(i); // сбрасывает i-й бит
bitvec.test(0); // возвращает false, поскольку первый бит сброшен
```

Оператор индексирования перегружается как константный. Константная версия возвращает логическое значение `true`, если бит по заданному индексу установлен, и значение `false` в противном случае. Неконстантная версия возвращает специальный тип, определенный классом `bitset`, позволяющий манипулировать битовым значением в позиции, заданной индексом:

```
bitvec[0] = 0;           // сбрасывает бит в позиции 0
bitvec[31] = bitvec[0]; // устанавливает последний бит в то же состояние,
                        // что и первый
bitvec[0].flip();       // инвертирует значение бита в позиции 0
~bitvec[0];             // эквивалентная операция; инвертирует бит
                        // в позиции 0
bool b = bitvec[0];     // преобразует значение bitvec[0] в тип bool
```

Возвращение значений из набора битов

Функции `to_ulong()` и `to_ullong()` возвращают значение, содержащее ту же битовую схему, что и объект класса `bitset`. Эти функции можно использовать, только если размер набора битов меньше или равен размеру типа `unsigned long` для функции `to_ulong()` и типа `unsigned long long` для функции `to_ullong()` соответственно:

```
unsigned long ulong = bitvec3.to_ulong();
cout << "ulong = " << ulong << endl;
```



Если значение в наборе битов не соответствует заданному типу, эти функции передают исключение `overflow_error` (см. раздел 5.6, стр. 257).

Операторы ввода-вывода типа `bitset`

Оператор ввода читает символы из входного потока во временный объект типа `string`. Чтение продолжается, пока не будет заполнен соответствующий набор битов, или пока не встретится символ, отличный от 1 или 0, или не встретится конец файла, или ошибка ввода. Затем этой временной строкой (см. раздел 17.2.1, стр. 907) инициализируется набор битов. Если прочитано меньше символов, чем насчитывает набор битов, старшие биты, как обычно, устанавливаются в 0.

Оператор вывода выводит битовую схему объекта `bitset`:

```
bitset<16> bits;
cin >> bits; // читать до 16 символов 1 или 0 из cin
cout << "bits: " << bits << endl; // вывести прочитанное
```

Использование наборов битов

Для иллюстрации применения наборов битов повторно реализуем код оценки из раздела 4.8 (стр. 213), использовавший тип `unsigned long` для представления результатов контрольных вопросов (сдал/не сдал) для 30 учеников:

```

bool status;
// версия, использующая побитовые операторы
unsigned long quizA = 0; // это значение используется как коллекция битов
quizA |= 1UL << 27;      // отметить ученика номер 27 как сдавшего
status = quizA & (1UL << 27); // проверить оценку ученика номер 27
quizA &= ~(1UL << 27);    // ученик номер 27 не сдал
// эквивалентные действия с использованием набора битов
bitset<30> quizB;         // зарезервировать по одному биту на студента; все
                          // биты инициализированы 0
quizB.set(27);            // отметить ученика номер 27 как сдавшего
status = quizB[27];       // проверить оценку ученика номер 27
quizB.reset(27);          // ученик номер 27 не сдал

```

Упражнения раздела 17.2.2

Упражнение 17.10. Используя последовательность 1, 2, 3, 5, 8, 13, 21, инициализируйте набор битов, у которого установлена 1 в каждой позиции, соответствующей числу в этой последовательности. Инициализируйте по умолчанию другой набор битов и напишите небольшую программу для установки каждого из соответствующих битов.

Упражнение 17.11. Определите структуру данных, которая содержит целочисленный объект, позволяющий отследить (сдал/не сдал) ответы на контрольную из 10 вопросов. Какие изменения (если они вообще понадобятся) необходимо внести в структуру данных, если в контрольной станет 100 вопросов?

Упражнение 17.12. Используя структуру данных из предыдущего вопроса, напишите функцию, получающую номер вопроса и значение, означающее правильный/неправильный ответ, и изменяющую результаты контрольной соответственно.

Упражнение 17.13. Создайте целочисленный объект, содержащий правильные ответы (да/нет) на вопросы контрольной. Используйте его для создания оценок контрольных вопросов для структуры данных из предыдущих двух упражнений.

17.3. Регулярные выражения

Регулярное выражение (regular expression) — это способ описания последовательности символов. Это чрезвычайно мощное средство программирования. Однако описание языков, используемых для определения регулярных выражений, выходит за рамки этой книги. Лучше сосредоточиться на использовании библиотеки регулярных выражений языка C++ (библиотеки RE), являющейся частью новой библиотеки. Библиотека RE определена в заголовке `regex` и задействует несколько компонентов, перечисленных в табл. 17.4.

Таблица 17.4. Компоненты библиотеки регулярных выражений

<code>Regex</code>	Класс, представляющий регулярное выражение
<code>regex_match()</code>	Сравнивает последовательность символов с регулярным выражением
<code>regex_search()</code>	Находит первую последовательность, соответствующую регулярному выражению
<code>regex_replace()</code>	Заменяет регулярное выражение, используя заданный формат
<code>sregex_iterator</code>	Адаптер итератора, вызывающий функцию <code>regex_search()</code> для перебора совпадений в строке
<code>smatch</code>	Класс контейнера, содержащего результаты поиска в строке
<code>ssub_match</code>	Результаты совпадения выражений в строке



Если вы еще не знакомы с использованием регулярных выражений, то имеет смысл просмотреть этот раздел и выяснить, на что способны регулярные выражения.

Класс `regex` представляет регулярное выражение. Кроме инициализации и присвоения, с классом `regex` допустимо немного операций. Они перечислены в табл. 17.6.

Функции `regex_match()` и `regex_search()` определяют, соответствует ли заданная последовательность символов предоставленному объекту класса `regex`. Функция `regex_match()` возвращает значение `true`, если вся исходная последовательность соответствует выражению; функция `regex_search()` возвращает значение `true`, если в исходной последовательности выражению соответствует подстрока. Есть также функция `regex_replace()`, описываемая в разделе 17.3.4 (стр. 929).

Аргументы функции `regex` описаны в табл. 17.5. Эти функции возвращают логическое значение и допускают перегрузку: одна версия получает дополнительный аргумент типа `smatch`. Если он есть, эти функции сохраняют дополнительную информацию об успехе обнаружения соответствия в предоставленном объекте класса `smatch`.

17.3.1. Использование библиотеки регулярных выражений

В качестве довольно простого примера рассмотрим поиск слов, нарушающих известное правило правописания “*i* перед *e*, кроме как после *c*”:

```
// найти символы ei, следующие за любым символом, кроме c
string pattern("[^c]ei");
// искомая схема должна присутствовать в целом слове
```

```
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern); // создать regex для поиска схемы
smatch results;   // определить объект для содержания результатов поиска
// определить строку, содержащую текст, соответствующий и не
// соответствующий схеме
string test_str = "receipt freind theif receive";
// использовать r для поиска соответствия в test_str
if (regex_search(test_str, results, r)) // если соответствие есть
cout << results.str() << endl;       // вывести соответствующее слово
```

Таблица 17.5. Аргументы функций `regex_search()` и `regex_match()`

Обратите внимание: функции возвращают логическое значение, означающее, было ли найдено соответствие.	
<code>(seq, m, r, mft)</code> <code>(seq, r, mft)</code>	Поиск регулярного выражения объекта <code>r</code> класса <code>regex</code> в символьной последовательности <code>seq</code> . Последовательность <code>seq</code> может быть строкой, парой итераторов, обозначающих диапазон, или указателем на символьный массив с нулевым символом в конце. <code>m</code> — это объект соответствия, используемый для хранения подробностей о соответствии. Типы объекта <code>m</code> и последовательности <code>seq</code> должны быть совместимы (см. раздел 17.3.1, стр. 919). <code>mft</code> — это необязательное значение <code>regex_constants::match_flag_type</code> . Это значение, описанное в табл. 17.13 (стр. 932), влияет на процесс поиска соответствия

Таблица 17.6. Операции с классом `regex` (и `wregex`)

<code>regex r(re)</code> <code>regex r(re, f)</code>	Параметр <code>re</code> представляет регулярное выражение и может быть строкой, парой итераторов, обозначающих диапазон символов, указателем на символьный массив с нулевым символом в конце, указателем на символ и количеством или списком символов в скобках. <code>f</code> — это флаги, определяющие выполнение объекта. Флаги <code>f</code> устанавливаются исходя из упомянутых ниже значений. Если флаги <code>f</code> не определены, по умолчанию применяется <code>ECMAScript</code>
<code>r1 = re</code>	Заменяет регулярное выражение в <code>r1</code> регулярным выражением <code>re</code> . <code>re</code> — это регулярное выражение, которое может быть другим объектом класса <code>regex</code> , строкой, указателем на символьный массив с нулевым символом в конце или списком символов в скобках
<code>r1.assign(re, f)</code>	То же самое, что и оператор присвоения (<code>=</code>). Параметр <code>re</code> и необязательный флаг <code>f</code> имеют тот же смысл, что и соответствующие аргументы конструктора <code>regex()</code>

<code>r.mark_count()</code>	Количество подвыражений (рассматриваются в разделе 17.3.3 (стр. 925)) в объекте <code>r</code>
<code>r.flags()</code>	Возвращает набор флагов для объекта <code>r</code>

Примечание: конструкторы и операторы присвоения могут передавать исключение типа `regex_error`.

Флаги, применяемые при определении объекта класса <code>regex</code> Определены в типах <code>regex</code> и <code>regex_constants::syntax_option_type</code>	
<code>icase</code>	Игнорировать регистр при поиске соответствия
<code>nosubs</code>	Не хранить соответствия подвыражений
<code>optimize</code>	Предпочтение скорости выполнения скорости создания
<code>ECMAScript</code>	Использование грамматики согласно ECMA-262
<code>basic</code>	Использование базовой грамматики регулярных выражений POSIX
<code>extended</code>	Использование расширенной грамматики регулярных выражения POSIX
<code>awk</code>	Использование грамматики POSIX версии языка <code>awk</code>
<code>grep</code>	Использование грамматики POSIX версии языка <code>grep</code>
<code>egrep</code>	Использование грамматики POSIX версии языка <code>egrep</code>

Начнем с определения строки для хранения искомого регулярного выражения. Регулярное выражение `[\^c]` означает любой символ, отличный от символа `'c'`, а `[\^c]ei` — любой такой символ, сопровождаемый символами `'ei'`. Эта схема описывает строки, содержащие только три символа. Необходимо найти целое слово, содержащее эту схему. Для соответствия слову необходимо регулярное выражение, которое будет соответствовать символам, расположенным прежде и после заданной трехсимвольной схемы.

Это регулярное выражение состоит из любого количества символов, сопровождаемых первоначальной трехсимвольной схемой и любым количеством дополнительных символов. По умолчанию объекты класса `regex` используют язык регулярных выражений `ECMAScript`. На языке `ECMAScript` схема `[[\alpha:]]` соответствует любому алфавитному символу, а символы `+` и `*` означают “один или несколько” и “нуль или более” соответственно. Таким образом, схема `[[\alpha:]]*` будет соответствовать любому количеству символов.

Регулярное выражение, сохраненное в строке `pattern`, используется для инициализации объекта `r` класса `regex`. Затем определяется строка, которая будет использована для проверки регулярного выражения. Строка `test_str` инициализируется словами, которые соответствуют схеме (например, “freund” и “theif”), и словами, которые ей не соответствуют (например, “receipt” и “receive”). Определим также объект `results` класса `smatch`, передаваемый

функции `regex_search()`. Если соответствие будет найдено, то объект `results` будет содержать подробности о том, где оно найдено.

Затем происходит вызов функции `regex_search()`. Если она находит соответствие, то возвращает значение `true`. Для вывода части строки `test_str`, соответствующей заданной схеме, используется функция-член `str()` объекта `results`. Функция `regex_search()` прекращает поиск, как только находит в исходной последовательности соответствующую подстроку. В результате вывод будет таким:

```
freind
```

Поиск всех соответствий во вводе представлен в разделе 17.3.2 (стр. 920).

Определение параметров объекта `regex`

При определении объекта класса `regex` или вызове его функции `assign()` для присвоения ему нового значения можно применить один или несколько флагов, влияющих на работу объекта класса `regex`. Эти флаги контролируют обработку, осуществляемую этим объектом. Последние шесть флагов, указанных в табл. 17.6, задают язык, на котором написано регулярное выражение. Установлен должен быть только один из флагов определения языка. По умолчанию установлен флаг `ECMAScript`, задающий использование объектом класса `regex` спецификации ECMA-262, являющейся языком регулярных выражений большинства веб-браузеров.

Другие три флага позволяют определять независимые от языка аспекты обработки регулярного выражения. Например, можно указать, что поиск регулярного выражения не будет зависеть от регистра символов.

В качестве примера используем флаг `icase` для поиска имен файлов с указанными расширениями. Большинство операционных систем распознают расширения без учета регистра символов: программа C++ может быть сохранена в файле с расширением `.cc`, `.Cc`, `.cC` или `.CC`. Давайте напишем регулярное выражение для распознавания любого из них наряду с другими общепринятыми расширениями файлов:

```
// один или несколько алфавитно-цифровых символов, сопровождаемых '.'
// и "cpp", "cxx" или "cc"
regex r("[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
smatch results;
string filename;
while (cin >> filename)
    if (regex_search(filename, results, r))
        cout << results.str() << endl; // вывод текущего соответствия
```

Это выражение будет соответствовать строке из одного или нескольких символов или цифр, сопровождаемых точкой и одним из трех расширений файла. Регулярное выражение будет соответствовать расширению файлов независимо от регистра.

Подобно тому, как специальные символы есть в языке C++ (см. раздел 2.1.3, стр. 70), у языков регулярных выражений, как правило, тоже есть специальные символы. Например, точка (.) обычно соответствует любому символу. Как и в языке C++, для обозначения специального характера символа его предваряют символом наклонной черты. Поскольку наклонная черта влево является также специальным символом в языке C++, в строковом литерале языка C++, означающем наклонную черту влево следует использовать вторую наклонную черту влево. Следовательно, чтобы представить точку в регулярном выражении, необходимо написать `\\. .`

Ошибки в определении и использовании регулярного выражения

Регулярное выражение можно считать самостоятельной “программой” на простом языке программирования. Этот язык не интерпретируется компилятором C++, и “компилируется” только во время выполнения, когда объект класса `regex` инициализируется или присваивается. Как и в любой написанной программе, в регулярных выражениях вполне возможны ошибки.



Важно понимать, что правильность синтаксиса регулярного выражения проверяется во время выполнения.

Если допустить ошибку в записи регулярного выражения, то передача исключения (см. раздел 5.6, стр. 257) типа `regex_error` произойдет только *во время выполнения*. Подобно всем стандартным типам исключений, у исключения `regex_error` есть функция `what()`, описывающая произошедшую ошибку (см. раздел 5.6.2, стр. 260). У исключения `regex_error` есть также функция-член `code()`, возвращающая числовой код (зависящий от реализации), соответствующий типу произошедшей ошибки. Стандартные сообщения об ошибках, которые могут быть переданы библиотекой RE, приведены в табл. 17.7.

Таблица 17.7. Причины ошибок в регулярном выражении

Определены в типах <code>regex</code> и <code>regex_constants::syntax_option_type</code>	
<code>error_collate</code>	Недопустимый запрос объединения элементов
<code>error_ctype</code>	Недопустимый класс символов
<code>error_escape</code>	Недопустимый управляющий или замыкающий символ
<code>error_backref</code>	Недопустимая обратная ссылка
<code>error_brack</code>	Несоответствие квадратных скобок (<code>[</code> или <code>]</code>)
<code>error_paren</code>	Несоответствие круглых скобок (<code>(</code> или <code>)</code>)

Определены в типах <code>regex</code> и <code>regex_constants::syntax_option_type</code>	
<code>error_brace</code>	Несоответствие фигурных скобок ({ или })
<code>error_badbrace</code>	Недопустимый диапазон в фигурных скобках ({ })
<code>error_range</code>	Недопустимый диапазон символов (например, [z-a])
<code>error_space</code>	Недостаточно памяти для выполнения этого регулярного выражения
<code>error_badrepeat</code>	Повторяющийся символ (* ?, + или { }) не предваряется допустимым регулярным выражением
<code>error_complexity</code>	Затребованное соответствие слишком сложно
<code>error_stack</code>	Недостаточно памяти для вычисления соответствия

Например, в схеме вполне можно пропустить по неосторожности скобку:

```
try {
    // ошибка: пропущена закрывающая скобка после alnum; конструктор
    // передаст исключение
    regex r("[[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
} catch (regex_error e)
{ cout << e.what() << "\ncode: " << e.code() << endl; }
```

При запуске на системе авторов эта программа выводит следующее:

```
regex_error(error_brack):
The expression contained mismatched [ and ].
code: 4
```

Компилятор определяет функцию-член `code()` для возвращения позиции ошибок, перечисленных в табл. 17.7, счет которых, как обычно, начинается с нуля.

СОВЕТ. ИЗБЕГАЙТЕ СОЗДАНИЯ НЕНУЖНЫХ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ

Как уже упоминалось, представляющая регулярное выражение “программа” компилируется во время выполнения, а не во время компиляции. Компиляция регулярного выражения может быть на удивление медленной операцией, особенно если используется расширенная грамматика регулярного выражения или выражение слишком сложно. В результате создание объекта класса `regex` и присвоение нового регулярного выражения уже существующему объекту класса `regex` может занять много времени. Для минимизации этих дополнительных затрат не создавайте больше объектов класса `regex`, чем необходимо. В частности, если регулярное выражение используется в цикле, его следует создать вне цикла, избежав перекомпиляции при каждой итерации.

**Классы регулярного выражения
и тип исходной последовательности**

Поиск возможен в любой из исходных последовательностей нескольких типов. Входные данные могут быть обычными символами типа `char` или `wchar_t`, и эти символы могут храниться в библиотечной строке или в массиве символов (или в его версии для `wchar_t`, или `wstring`). Библиотека RE определяет отдельные типы, соответствующие этим разным типам исходных последовательностей.

Предположим, например, что класс `regex` содержит регулярное выражение типа `char`. Для типа `wchar_t` библиотека определяет также класс `wregex`, поддерживающий все операции класса `regex`. Единственное различие в том, что инициализаторы класса `wregex` должны использовать тип `wchar_t` вместо типа `char`.

Типы соответствий и итераторов (они рассматриваются в следующих разделах) более специфичны. Они отличаются не только типом символов, но и тем, является ли последовательность библиотечным типом или массивом: класс `smatch` представляет исходные последовательности типа `string`; класс `cmatch` — символьные массивы; `wsmatch` — строки Unicode (`wstring`); `wcmatch` — массивы символов `wchar_t`.

Таблица 17.8. Библиотечные классы регулярных выражений

Тип исходной последовательности	Используемый класс регулярного выражения
<code>string</code>	<code>regex</code> , <code>smatch</code> , <code>ssub_match</code> и <code>sregex_iterator</code>
<code>const char*</code>	<code>regex</code> , <code>cmatch</code> , <code>csub_match</code> и <code>cregex_iterator</code>
<code>wstring</code>	<code>wregex</code> , <code>wsmatch</code> , <code>wssub_match</code> и <code>wsregex_iterator</code>
<code>const wchar_t*</code>	<code>wregex</code> , <code>wcmatch</code> , <code>wcsub_match</code> и <code>wcregex_iterator</code>

Важный момент: используемый тип библиотеки RE должен соответствовать типу исходной последовательности. Соответствие классов видам исходных последовательностей приведено в табл. 17.8. Например:

```
regex r("[[:alnum:]]+\\.(cpp|cxx|cc)$", regex::icase);
smatch results; // будет соответствовать последовательности типа string,
                // но не char*
if (regex_search("myfile.cc", results, r)) // ошибка: ввод char*
    cout << results.str() << endl;
```

Компилятор C++ отклонит этот код, поскольку тип аргумента и тип исходной последовательности не совпадают. Если необходимо искать в символьном массиве, то следует использовать объект класса `smatch`:

```
smatch results; // будет соответствовать последовательности символьного
                // массива
if (regex_search("myfile.cc", results, r))
    cout << results.str() << endl; // вывод текущего соответствия
```

Обычно программы используют исходные последовательности типа `string` и соответствующие ему версии компонентов библиотеки RE.

Упражнения раздела 17.3.1

Упражнение 17.14. Напишите несколько регулярных выражений, предназначенных для создания различных ошибок. Запустите программу и посмотрите, какие сообщения выводит ваш компилятор для каждой ошибки.

Упражнение 17.15. Напишите программу, используя схему поиска слов, нарушающих правило “*i* перед *e*, кроме как после *c*”. Организуйте приглашение для ввода пользователем слова и вывод результата его проверки. Проверьте свою программу на примере слов, которые нарушают и не нарушают это правило.

Упражнение 17.16. Что будет при инициализации объекта класса `regex` в предыдущей программе значением “`^[^c]ei`”? Проверьте свою программу, используя эту схему, и убедитесь в правильности своих ожиданий.

17.3.2. Типы итераторов классов соответствия и `regex`

Программа проверки правила “*i* перед *e*, кроме как после *c*” на стр. 913 выводила только первое соответствие в исходной последовательности. Используя итератор `sregex_iterator`, можно получить все соответствия. Итераторы класса `regex` являются адаптерами итератора (см. раздел 9.6, стр. 473), привязанные к исходной последовательности и объекту класса `regex`. Как было описано в табл. 17.8, для каждого типа исходной последовательности используется специфический тип итератора. Операции с итераторами описаны в табл. 17.9.

Когда итератор `sregex_iterator` связывается со строкой и объектом класса `regex`, итератор автоматически позиционируется на первое соответствие в заданной строке. Таким образом, конструктор `sregex_iterator()` вызывает функцию `regex_search()` для данной строки и объекта класса `regex`. При обращении к значению итератора возвращается объект класса `smatch`, соответствующий результатам самого последнего поиска. При приращении итератора для поиска следующего соответствия в исходной строке вызывается функция `regex_search()`.

Таблица 17.9. Операции с итератором `sregex_iterator`

Эти операции применимы также к итераторам <code>sregex_iterator</code> , <code>wsregex_iterator</code> и <code>wcregex_iterator</code>	
<code>sregex_iterator it(b, e, r);</code>	<code>it</code> — это итератор <code>sregex_iterator</code> , перебирающий строку, обозначенную итераторами <code>b</code> и <code>e</code> . Вызов <code>regex_search(b, e, r)</code> устанавливает итератор <code>it</code> на первое соответствие во вводе
<code>sregex_iterator end;</code>	Итератор <code>sregex_iterator</code> , указывающий на позицию после конца
<code>*it</code> <code>it-></code>	Возвращает ссылку на объект класса <code>smatch</code> или указатель на объект класса <code>smatch</code> от самого последнего вызова функции <code>regex_search()</code>
<code>++it</code> <code>it++</code>	Вызывает функцию <code>regex_search()</code> для исходной последовательности, начиная сразу после текущего соответствия. Префиксная версия возвращает ссылку на приращенный итератор, а постфиксная возвращает прежнее значение
<code>it1 == it2</code> <code>it1 != it2</code>	Два итератора <code>sregex_iterator</code> равны, если оба они итераторы после конца. Два не конечных итератора равны, если они созданы из той же исходной последовательности и объекта класса <code>regex</code>

Использование итератора `sregex_iterator`

В качестве примера дополним программу поиска нарушения правила “*i* перед *e*, кроме как после *c*” в текстовом файле. Подразумевается, что `file` класса `string` содержит все содержимое исходного файла, на котором осуществляется поиск. Новая версия программы будет использовать ту же схему, что и ранее, но для поиска применим итератор `sregex_iterator`:

```
// найти символы ei, следующие за любым символом, кроме c
string pattern("[^c]ei");
// искомая схема должна присутствовать в целом слове
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern, regex::icase); // игнорируем случай выполнения
                               // соответствия
// будет последовательно вызывать regex_search() для поиска всех
// соответствий в файле
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it)
    cout << it->str() << endl; // соответствующее слово
```

Цикл `for` перебирает все соответствия `r` в строке `file`. Инициализатор в цикле `for` определяет итераторы `it` и `end_it`. При определении итератора `it` конструктор `sregex_iterator()` вызывает функцию `regex_search()` для позиционирования итератора `it` на первое соответствие в строке `file`.

Пустой итератор `sregex_iterator`, `end_it` действует как итератор после конца. Приращение в цикле `for` “перемещает” итератор, вызвав функцию `regex_search()`. При обращении к значению итератора возвращается объект класса `smatch`, представляющий текущее соответствие. Для вывода соответствующего слова вызывается функция-член `str()`.

Данный цикл `for` как бы перепрыгивает с одного соответствия на другое, как показано на рис. 17.1.

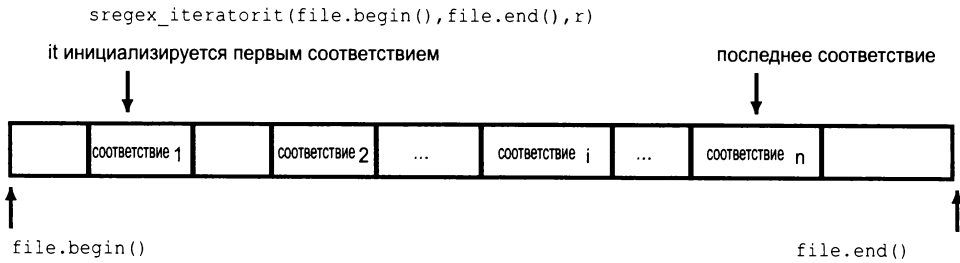


Рис. 17.1. Использование итератора `sregex_iterator`

Использование данных соответствия

Если запустить этот цикл для строки `test_str` из первоначальной программы, вывод был бы таким:

```
Freind
theif
```

Однако вывод только самого слова, соответствующего заданному выражению, не очень полезен. При запуске программы для большой исходной последовательности, например для текста этой главы, имело бы смысл увидеть контекст, в котором встретилось слово. Например:

```
hey read or write according to the type
>>> being <<<
handled. The input operators ignore whi
```

Кроме возможности вывода части исходной строки, в которой встретилось соответствие, классы соответствия предоставляют более подробную информацию о соответствии. Возможные операции с этими типами перечислены в табл. 17.10 и 17.11.

Более подробная информация о `smatch` и `ssub_match` приведена в следующем разделе, а пока достаточно знать, что они предоставляют доступ к контексту соответствия. У типов соответствия есть функции-члены `prefix()` и `suffix()`, возвращающие объект класса `ssub_match`, представляющий часть исходной последовательности перед и после текущего соответствия соответственно. У класса `ssub_match` есть функции-члены `str()` и `length()`, возвращающие соответствующую строку и ее размер соответственно. Используя эти функции, можно переписать цикл программы проверки правописания:

```
// тот же заголовок цикла for, что и прежде
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it) {
    auto pos = it->prefix().length(); // размер префикса
    pos = pos > 40 ? pos - 40 : 0;    // необходимо до 40 символов
    cout << it->prefix().str().substr(pos) // последняя часть префикса
         << "\n\t\t\t" << it->str() << " <<<\n" // соответствующее
                                                // слово
         << it->suffix().str().substr(0, 40) // первая часть суффикса
         << endl;
}
```

Таблица 17.10. Операции с типом `smatch`

Эти операции применимы также к типам <code>cmatch</code>, <code>wsmatch</code>, <code>wcmatch</code> и соответствующим типам <code>csub_match</code>, <code>wssub_match</code> и <code>wcsub_match</code>.	
<code>m.ready()</code>	Возвращает значение <code>true</code> , если <code>m</code> был установлен вызовом функции <code>regex_search()</code> или <code>regex_match()</code> , в противном случае — значение <code>false</code> (в этом случае результат операции с <code>m</code> непредсказуем)
<code>m.size()</code>	Возвращает значение 0, если соответствия не найдено, в противном случае — на единицу больше, чем количество подвыражений в последнем соответствующем регулярном выражении
<code>m.empty()</code>	Возвращает значение <code>true</code> , если размер нулевой
<code>m.prefix()</code>	Возвращает объект класса <code>ssub_match</code> , представляющий последовательность перед соответствием
<code>m.suffix()</code>	Возвращает объект класса <code>ssub_match</code> , представляющий часть после конца соответствия
<code>m.format(...)</code>	См. табл. 17.12 (стр. 930)
В функциях, получающих индекс, <code>n</code> по умолчанию имеет значение нуль и должно быть меньше <code>m.size()</code>. Первое соответствие (с индексом 0) представляет общее соответствие.	
<code>m.length(n)</code>	Возвращает размер соответствующего подвыражения номер <code>n</code>
<code>m.position(n)</code>	Дистанция подвыражения номер <code>n</code> от начала последовательности
<code>m.str(n)</code>	Соответствующая строка для подвыражения номер <code>n</code>
<code>m[n]</code>	Объект <code>ssub_match</code> , соответствующий подвыражению номер <code>n</code>
<code>m.begin(), m.end()</code> <code>m.cbegin(), m.cend()</code>	Итераторы элементов <code>sub_match</code> в <code>m</code> . Как обычно, функции <code>cbegin()</code> и <code>cend()</code> возвращают итераторы <code>const_iterator</code>

Более подробная информация о `smatch` и `ssub_match` приведена в следующем разделе, а пока достаточно знать, что они предоставляют доступ к контексту соответствия. У типов соответствия есть функции-члены `prefix()` и `suffix()`, возвращающие объект класса `ssub_match`, представляющий часть исходной последовательности перед и после текущего соответствия соответственно. У класса `ssub_match` есть функции-члены `str()` и `length()`, возвращающие соответствующую строку и ее размер соответственно. Используя эти функции, можно переписать цикл программы проверки правописания:

```
// тот же заголовок цикла for, что и прежде
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it) {
    auto pos = it->prefix().length(); // размер префикса
    pos = pos > 40 ? pos - 40 : 0;    // необходимо до 40 символов
    cout << it->prefix().str().substr(pos) // последняя часть префикса
         << "\n\t\t>>> " << it->str() << " <<<\n" // соответствующее
                                     // слово
         << it->suffix().str().substr(0, 40) // первая часть суффикса
         << endl;
}
```

Сам цикл работает, как и прежде. Изменился процесс в цикле `for`, представленный на рис. 17.2.

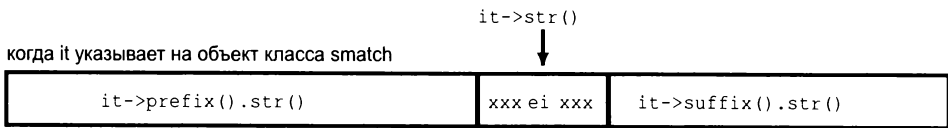


Рис. 17.2. Объект класса `smatch`, представляющий некое соответствие

Здесь происходит вызов функции `prefix()`, возвращающий объект класса `ssub_match`, представляющий часть строки `file` перед текущим соответствием. Чтобы выяснить, сколько символов находится в части строки `file` перед соответствием, вызовем функцию `length()` для этого объекта класса `ssub_match`. Затем скорректируем значение `pos` так, чтобы оно было индексом 40-го символа от конца префикса. Если у префикса меньше 40 символов, устанавливаем `pos` в 0, означая, что выведен весь префикс. Функция `substr()` (см. раздел 9.5.1, стр. 463) используется для вывода от данной позиции до конца префикса.

После вывода символов, предшествующих соответствию, выводится само соответствие с некоторым дополнительным оформлением, чтобы соответствующее слово выделилось в выводе. После вывода соответствующей части выводится до 40 следующих после соответствия символов строки `file`.

Упражнения раздела 17.3.2

Упражнение 17.17. Измените свою программу так, чтобы она находила все слова в исходной последовательности, нарушающие правило “*i* перед *e*, кроме как после *c*”.

Упражнение 17.18. Пересмотрите свою программу так, чтобы игнорировать слова, содержащие сочетание “*ei*”, но не являющиеся ошибочными, такие как “*albeit*” и “*neighbor*”.

17.3.3. Использование подвыражений

Схема в регулярном выражении зачастую содержит одно или несколько *подвыражений* (subexpression). Подвыражение — это часть схемы, которая сама имеет значение. Для обозначения подвыражения в регулярном выражении, как правило, используют круглые скобки.

Например, в схеме для поиска соответствий расширений файлов языка C++ (см. раздел 16.1.2, стр. 915) круглые скобки используются для группировки возможных расширений. Каждый раз, когда альтернативы группируются с использованием круглых скобок, одновременно объявляется, что эти альтернативы формируют подвыражение. Это выражение можно переписать так, чтобы оно предоставило доступ к имени файла, являющемуся той частью схемы, которая предшествует точке:

```
// r содержит два подвыражения: первое - часть имени файла перед точкой,  
// второе - расширение файла  
regex r("([[:alnum:]]+)\. (cpp|cxx|cc)$", regex::icase);
```

Теперь в схеме два заключенных в скобки подвыражения:

- `([[:alnum:]]+)` — представляет последовательность из одного или нескольких символов;
- `(cpp| cxx| cc)` — представляет расширения файлов.

Теперь программу из раздела 16.1.2 (стр. 915) можно переписать так (изменив оператора вывода), чтобы выводить только имя файла:

```
if (regex_search(filename, results, r))  
    cout << results.str(1) << endl; // вывести первое подвыражение
```

В первоначальной программе для поиска схемы `r` в строке `filename` использовался вызов функции `regex_search()`, а также объект `results` класса `smatch` для содержания результата поиска соответствия. Если вызов успешен, выводится результат. Но в этой программе выводится `str(1)`, т.е. соответствие для первого подвыражения.

Кроме информации об общем соответствии, объекты соответствия предоставляют доступ к каждому соответствию подвыражению в схеме. К соответст-

виям подвыражению обращаются по позиции. Первое соответствие подвыражению, расположенное в позиции 0, представляет соответствие для всей схемы. После него располагается каждое подвыражение. Следовательно, имя файла, являющееся первым подвыражением в схеме, находится в позиции 1, а расширение файла — в позиции 2.

Например, если именем файла будет `foo.cpp`, то `results.str(0)` содержит строку `"foo.cpp"`; `results.str(1)` — `"foo"`, а `results.str(2)` — `".cpp"`. В этой программе требуется часть имени перед точкой, что является первым подвыражением, поэтому следует вывести `results.str(1)`.

Подвыражения для проверки правильности данных

Подвыражения обычно используются для проверки данных, которые должны соответствовать некоему определенному формату. Например, в Америке номера телефонов имеют десять цифр, включая код города и местный номер из семи цифр. Код города зачастую, но не всегда, заключен в круглые скобки. Остальные семь цифр могут быть отделены тире, точкой или пробелом либо не отделяться вообще. Данные в некоторых из этих форматов могли бы быть приемлемы, а в других — нет. Процесс будет состоять из двух этапов: сначала используем регулярное выражение для поиска последовательностей, которые могли бы быть номерами телефонов, а затем вызовем функцию для окончательной проверки правильности данных.

Прежде чем написать схему номеров телефона, необходимо рассмотреть еще несколько аспектов языка регулярных выражений на языке ECMAScript.

- `\{d}` представляет одиночную цифру, а `\{d\}{n}` — последовательность из `n` цифр. (Например, `\{d\}{3}` соответствует последовательности из трех цифр.)
- Набор символов в квадратных скобках позволяет задать соответствие любому из трех символов. (Например, `[-.]` соответствует тире, точке или пробелу. Обратите внимание: у точки в квадратных скобках нет никакого специального смысла.)
- Компонент, следующий за символом `'?'`, не обязательный. (Например, `\{d\}{3}[-. ]?\{d\}{4}` соответствует трем цифрам, сопровождаемым опциональными тире, точкой или пробелом и еще четырьмя цифрами. Этой схеме соответствовало бы 555-0132, или 555.0132, или 555 0132, или 5550132).
- Как и в языке C++, в ECMAScript символ за наклонной чертой означает, что он представляет себя, а не специальное значение. Поскольку данная схема включает круглые скобки, являющиеся специальными символами в языке ECMAScript, круглые скобки, являющиеся частью схемы, следует представить как `\ (` или `\ )`.

Поскольку наклонная черта влево является специальным символом в языке C++, когда он встречается в схеме, следует добавить вторую наклонную черту, чтобы указать языку C++, что имеется в виду символ \. Следовательно, чтобы представить регулярное выражение `\{d\}{3}`, нужно написать `\\{d\}{3}`.

Для проверки номеров телефонов следует обратиться к компонентам схемы. Например, необходимо проверить, что если номер использует открывающую круглую скобку для кода города, то он использует также закрывающую скобку после него. В результате такой номер, как (908.555.1800, следует отклонить.

Для определения такого соответствия необходимо регулярное выражение, использующее подвыражения. Каждое подвыражение заключается в пару круглых скобок:

```
// все выражение состоит из семи подвыражений: ( ddd ) разделитель ddd
// разделитель dddd
// подвыражения 1, 3, 4 и 6 опциональны; а 2, 5 и 7 содержат цифры
"(\()?(\\d{3})\\)\)?([- ])?(\\d{3})([- ])?(\\d{4})";
```

Поскольку схема использует круглые скобки, а также из-за использования наклонных черт, эту схему трудно прочесть (и написать!). Проще всего прочитать ее по каждому отдельному (заключенному в скобки) подвыражению.

1. `(\()?` необязательная открывающая скобка для кода города.
2. `(\\d{3})` код города.
3. `\\)` необязательная закрывающая скобка для кода города.
4. `([- ])?` необязательный разделитель после кода города.
5. `(\\d{3})` следующие три цифры номера.
6. `([- ])?` другой необязательный разделитель.
7. `(\\d{4})` последние четыре цифры номера.

Следующий код использует эту схему для чтения файла и находит данные, соответствующие общей схеме телефонных номеров. Для проверки допустимости формата номеров используется функция `valid()`:

```
string phone =
    "(\()?(\\d{3})\\)\)?([- ])?(\\d{3})([- ])?(\\d{4})";
regex r(phone); // объект regex для поиска схемы
smatch m;
string s;
// прочитать все записи из входного файла
while (getline(cin, s)) {
    // для каждого подходящего номера телефона
    for (sregex_iterator it(s.begin(), s.end(), r), end_it;
         it != end_it; ++it)
        // проверить допустимость формата номера
        if (valid(*it))
```

```

        cout << "valid: " << it->str() << endl;
    else
        cout << "not valid: " << it->str() << endl;
}

```

Операции с типом соответствия

Напишем функцию `valid()`, используя операции типа соответствия, приведенные в табл. 17.11. Не следует забывать, что схема `pattern` состоит из семи подвыражений. В результате каждый объект класса `smatch` будет содержать восемь элементов `ssub_match`. Элемент `[0]` представляет общее соответствие, а элементы `[1] – [7]` представляют каждое из соответствующих подвыражений.

Таблица 17.11. Операции с типом соответствия

Эти операции применимы к типам <code>ssub_match</code> , <code>csub_match</code> , <code>wssub_match</code> и <code>wcsub_match</code>	
<code>matched</code>	Открытая логическая переменная-член, означающая соответствие объекта класса <code>ssub_match</code>
<code>first</code> <code>second</code>	Открытые переменные-члены, являющиеся итераторами на начало последовательности соответствия и ее следующий элемент после последнего. Если соответствия нет, то <code>first</code> и <code>second</code> равны
<code>length()</code>	Размер текущего объекта соответствия. Возвращает 0, если переменная-член <code>matched</code> содержит значение <code>false</code>
<code>str()</code>	Возвращает строку, содержащую соответствующую часть ввода. Возвращает пустую строку, если переменная-член <code>matched</code> содержит значение <code>false</code>
<code>s = ssub</code>	Преобразует объект <code>ssub</code> класса <code>ssub_match</code> в строку <code>s</code> . Эквивалент вызова <code>s = ssub.str()</code> . Оператор преобразования не является явным (см. раздел 14.9.1, стр. 735)

Когда происходит вызов функции `valid()`, известно, что общее соответствие имеется, но неизвестно, какие из необязательных подвыражений являются частью этого соответствия. Переменная-член `matched` класса `ssub_match`, соответствующая определенному подвыражению, содержит значение `true`, если это подвыражение является частью общего соответствия.

В правильном номере телефона код города либо полностью заключается в скобки, либо не заключается в них вообще. Поэтому действие функции `valid()` зависит от того, начинается ли номер с круглой скобки или нет:

```

bool valid(const smatch& m)
{
    // если перед кодом города есть открывающая скобка
    if(m[1].matched)

```

```

// за кодом города должна быть закрывающая скобка
// и остальная часть номера непосредственно или через пробел
return m[3].matched
    && (m[4].matched == 0 || m[4].str() == " ");
else
    // здесь после кода города не может быть закрывающей скобки
    // но разделители между другими двумя компонентами должны быть
    // корректны
    return !m[3].matched
        && m[4].str() == m[6].str();
}

```

Начнем с проверки соответствия первому подвыражению (т.е. открывающей скобки). Это подвыражение находится в элементе `m[1]`. Если это соответствие есть, то номер начинается с открывающей скобки. В таком случае номер будет допустимым, только если подвыражение после кода города также будет соответствующим (т.е. будет закрывающая скобка после кода города). Кроме того, если скобки в начале номера корректны, то следующим символом должен быть пробел или первая цифра следующей части номера.

Если элемент `m[1]` не соответствует (т.е. открывающей скобки нет), то подвыражение после кода города также должно быть пустым. Если это так и если остальные разделители совпадают, то номер допустим, но не в противном случае.

Упражнения раздела 17.3.3

Упражнение 17.19. Почему можно вызывать функцию `m[4].str()` без предварительной проверки соответствия элемента `m[4]`?

Упражнение 17.20. Напишите собственную версию программы для проверки номеров телефонов.

Упражнение 17.21. Перепишите программу номеров телефонов из раздела 8.3.2 (стр. 416) так, чтобы использовать функцию `valid()`, определенную в этом разделе.

Упражнение 17.22. Перепишите программу номеров телефонов так, чтобы она позволила разделять три части номера телефона любыми символами.

Упражнение 17.23. Напишите регулярное выражение для поиска почтовых индексов. У них может быть пять или девять цифр. Первые пять цифр могут быть отделены от остальных четырех тире.

17.3.4. Использование функции `regex_replace()`

Регулярные выражения зачастую используются не только для поиска, но и для замены одной последовательности другой. Например, может потребоваться преобразовать американские номера телефонов в формат "ddd.ddd.dddd", где код города и три последующие цифры разделены точками.

Когда необходимо найти и заменить регулярное выражение в исходной последовательности, используется функция `regex_replace()`. Подобно функции поиска, функция `regex_replace()`, описанная в табл. 17.12, получает входную символьную последовательность и объект класса `regex`. Следует также передать строку, которая описывает необходимый вывод.

Таблица 17.12. Функции замены регулярного выражения

<code>m.format(dest, fmt, mft)</code> <code>m.format(fmt, mft)</code>	Создает форматированный вывод, используя формат строки <i>fmt</i> , соответствие в <i>m</i> и необязательные флаги <code>match_flag_type</code> в <i>mft</i> . Первая версия пишет в итератор вывода <i>dest</i> (см. раздел 10.5.1, стр. 525) и получает формат <i>fmt</i> , который может быть строкой или парой указателей, обозначающих диапазон в символьном массиве. Вторая версия возвращает строку, которая содержит вывод и получает формат <i>fmt</i> , являющийся строкой или указателем на символьный массив с нулевым символом в конце. По умолчанию <i>mft</i> имеет значение <code>format_default</code>
<code>regex_replace</code> (<i>dest, seq, r, fmt, mft</i>) <code>regex_replace</code> (<i>seq, r, fmt, mft</i>)	Перебирает последовательность <i>seq</i> , используя функцию <code>regex_search()</code> для поиска соответствий объекту <i>r</i> класса <code>regex</code> . Использует формат строки <i>fmt</i> и необязательные флаги <code>match_flag_type</code> в <i>mft</i> для формирования вывода. Первая версия пишет в итератор вывода <i>dest</i> и получает пару итераторов для обозначения последовательности <i>seq</i> . Вторая возвращает строку, содержащую вывод, а <i>seq</i> может быть строкой или указателем на символьный массив с нулевым символом в конце. Во всех случаях формат <i>fmt</i> может быть строкой или указателем на символьный массив с нулевым символом в конце. По умолчанию <i>mft</i> имеет значение <code>match_default</code>

Строку замены составляют подлежащие включению символы вместе с подвыражениями из соответствующей подстроки. В данном случае следует использовать второе, пятое и седьмое подвыражения из строки замены. Первое, третье, четвертое и шестое подвыражения игнорируются, поскольку они использовались в первоначальном форматировании номера, но не являются частью формата замены. Для ссылки на конкретное подвыражение используется символ `$`, сопровождаемый индексом подвыражения:

```
string fmt = "$2.$5.$7"; // переформатировать номера в ddd.ddd.dddd
```

Схему регулярного выражения и строку замены можно использовать следующим образом:

```
regex r(phone); // regex для поиска схемы
string number = "(908) 555-1800";
cout << regex_replace(number, r, fmt) << endl;
```

Вывод этой программы будет таким:

```
908.555.1800
```

Замена только части исходной последовательности

Куда интересней использование обработки регулярных выражений для замены номеров телефонов в большом файле. Предположим, например, что имеется файл имен и номеров телефонов, содержащий такие данные:

```
morgan (201) 555-2368 862-555-0123/
drew (973)555.0130
lee (609) 555-0132 2015550175 800.555-0000
```

Их следует преобразовать в такой формат:

```
morgan 201.555.2368 862.555.0123
drew 973.555.0130
lee 609.555.0132 201.555.0175 800.555.0000
```

Это преобразование можно осуществить следующим образом:

```
int main()
{
    string phone =
        "\\(\\)?(\\d{3}) (\\d{3})?([- .])?(\\d{3}) ([- .])?(\\d{4})";
    regex r(phone); // regex для поиска схемы
    smatch m;
    string s;
    string fmt = "$2.$5.$7"; // переформатировать номера в ddd.ddd.dddd
    // прочитать каждую запись из входного файла
    while (getline(cin, s))
        cout << regex_replace(s, r, fmt) << endl;
    return 0;
}
```

Каждая запись читается в строку `s` и передается функции `regex_replace()`. Эта функция находит и преобразует все соответствия исходной последовательности.

Флаги, контролирующие соответствия и формат

Кроме флагов обработки регулярных выражений, библиотека определяет также флаги, позволяющие контролировать процесс поиска соответствия и форматирования при замене. Их значения приведены в табл. 17.13. Эти флаги могут быть переданы функции `regex_search()`, или функции `regex_match()`, или функциям-членам формата класса `smatch`.

Таблица 17.13. Флаги соответствия

Определено в <code>regex_constants::match_flag_type</code>	
<code>match_default</code>	Эквивалент <code>format_default</code>
<code>match_not_bol</code>	Не рассматривать первый символ как начало строки
<code>match_not_eol</code>	Не рассматривать последний символ как конец строки
<code>match_not_bow</code>	Не рассматривать первый символ как начало слова
<code>match_not_eow</code>	Не рассматривать последний символ как конец слова
<code>match_any</code>	Если соответствий несколько, может быть возвращено любое из них
<code>match_not_null</code>	Не соответствует пустой последовательности
<code>match_continuous</code>	Соответствие должно начинаться с первого символа во вводе
<code>match_prev_avail</code>	У исходной последовательности есть символы перед первым
<code>format_default</code>	Строка замены использует правила ECMAScript
<code>format_sed</code>	Строка замены использует правила POSIX sed
<code>format_no_copy</code>	Не выводить несоответствующие части ввода
<code>format_first_only</code>	Заменить только первое вхождение

Флаги соответствия и формата имеют тип `match_flag_type`. Их значения определяются в пространстве имен `regex_constants`. Подобно пространству имен `placeholders`, используемому с функциями `bind()` (см. раздел 10.3.4, стр. 511), пространство имен `regex_constants` определено в пространстве имен `std`. Для использования имени из пространства `regex_constants` его следует квалифицировать именами обоих пространств имен:

```
using std::regex_constants::format_no_copy;
```

Это объявление указывает, что когда код использует флаг `format_no_copy`, необходим объект из пространства имен `std::regex_constants`. Вместо этого можно использовать и альтернативную форму `using`, рассматриваемую в разделе 18.2.2 (стр. 990):

```
using namespace std::regex_constants;
```

Использование флагов формата

По умолчанию функция `regex_replace()` выводит всю исходную последовательность. Части, которые не соответствуют регулярному выражению, выводятся без изменений, а соответствующие части оформляются, как указано строкой формата. Это стандартное поведение можно изменить, указав флаг `format_no_copy` в вызове функции `regex_replace()`:


```
// выдать только номера телефона: используется новая строка формата
string fmt2 = "$2.$5.$7 "; // поместить пробел как разделитель после
                          // последнего числа
// указать regex_replace() копировать только заменяемый текст
cout << regex_replace(s, r, fmt2, format_no_copy) << endl;
```

С учетом того же ввода эта версия программы создает такой вывод:

```
201.555.2368 862.555.0123
973.555.0130
609.555.0132 201.555.0175 800.555.0000
```

Упражнения раздела 17.3.4

Упражнение 17.24. Напишите собственную версию программы для переформатирования номеров телефонов.

Упражнение 17.25. Перепишите свою программу телефонных номеров так, чтобы она выводила только первый номер для каждого человека.

Упражнение 17.26. Перепишите свою программу телефонных номеров так, чтобы она выводила только второй и последующие номера телефонов для людей с несколькими номерами телефонов.

Упражнение 17.27. Напишите программу, которая переформатировала бы почтовый индекс с девятью цифрами как dddd-dddd.

17.4. Случайные числа

**C++
11** Программы нередко нуждаются в источнике случайных чисел. До нового стандарта языка C и C++ полагались на простую библиотечную функцию языка C по имени `rand()`. Эта функция создает псевдослучайные целые числа, равномерно распределенные в диапазоне от нуля до зависимого от системы максимального значения, которое по крайней мере не меньше 32767.

У функции `rand()` несколько проблем: многим, если не всем, программам нужны случайные числа в совершенно другом диапазоне, отличном от используемого функцией `rand()`. Некоторые приложения требуют случайных чисел с плавающей запятой, другим нужны числа с неоднородным распределением. Когда разработчики пытаются преобразовывать диапазон, тип или распределение чисел, созданных функцией `rand()`, их случайность зачастую теряется.

Библиотека случайных чисел, определенная в заголовке `random`, решает эти проблемы за счет набора взаимодействующих классов: классов *процессора случайных чисел* (random-number engine) и классов *распределения случайного числа* (random-number distribution). Эти классы описаны в табл. 17.14. Процес-

сор создает последовательность беззнаковых случайных чисел, а распределение использует процессор для создания случайных чисел определенного типа в заданном диапазоне, распределенном согласно указанному вероятностному распределению.

Таблица 17.14. Компоненты библиотеки случайных чисел

Процессор	Типы, создающие последовательность случайных беззнаковых целых чисел
Распределение	Типы, использующие процессор для возвращения чисел согласно заданному распределению вероятности



Программы C++ больше не должны использовать библиотечную функцию `rand()`. Для этого следует использовать класс `default_random_engine` наряду с соответствующим объектом распределения.

17.4.1. Процессоры случайных чисел и распределения

Процессоры случайных чисел — это классы объектов функции (см. раздел 14.8, стр. 722), определяющие оператор вызова, не получающий никаких аргументов и возвращающий случайное беззнаковое число. Вызвав объект типа процессора случайных чисел, можно получить простые случайные числа:

```
default_random_engine e; // создает случайное беззнаковое число
for (size_t i = 0; i < 10; ++i)
    // e() "вызывает" объект для создания следующего случайного числа
    cout << e() << " ";
```

На системе авторов эта программа выводит:

16807 282475249 1622650073 984943658 1144108930 470211272 ...

Здесь был определен объект `e` типа `default_random_engine`. В цикле `for` происходит вызов объекта `e`, возвращающий следующее случайное число.

Библиотека определяет несколько процессоров случайных чисел, отличающихся производительностью и качеством случайности. Каждый компилятор определяет один из этих процессоров как *стандартный процессор случайных чисел* (`default random engine`) (тип `default_random_engine`). Этот тип предназначен для процессоров с наиболее общеприменимыми свойствами (табл. 17.15). Список типов и функций процессоров, определенных стандартом, приведен в разделе А.3.2 (стр. 1100).

В большинстве случаев вывод процессора сам по себе непригоден для использования, поскольку, как уже упоминалось, это простые случайные числа. Проблема в том, что эти числа обычно охватывают диапазон, отличный от необходимого. *Правильное* преобразование диапазона случайного числа на удивление трудно.

Типы распределения и процессоры

Чтобы получить число в определенном диапазоне, используется объект типа `распределения`:

```
// однородное распределение от 0 до 9 включительно
uniform_int_distribution<unsigned> u(0,9);
default_random_engine e; // создает случайные беззнаковые целые числа
for (size_t i = 0; i < 10; ++i)
// u использует e как источник чисел
// каждый вызов возвращает однородно распределенное значение в заданном
// диапазоне
cout << u(e) << " ";
```

Вывод таков:

```
0 1 7 4 5 2 0 6 6 9
```

Здесь `u` определяется как объект типа `uniform_int_distribution<unsigned>`. Этот тип создает однородно распределенные беззнаковые значения. При определении объекта этого типа можно задать минимум и максимум необходимых значений. Определение `u(0, 9)` указывает, что необходимы числа в диапазоне от 0 до 9 *включительно*. Распределение случайного числа использует включающие диапазоны, позволяющие получить любое возможное целочисленное значение в нем.

Подобно типам процессоров, типы распределения также являются классами объектов функции. Типы распределения определяют оператор вызова, получающий процессор случайных чисел как аргумент. Объект распределения использует свой аргумент процессора для создания случайного числа, которое объект распределения сопоставит с определенным распределением.

Обратите внимание на то, что объект процессора передается непосредственно, `u(e)`. Если бы вызов был написан как `u(e())`, то произошла бы попытка передать следующее созданное `e` значение в `u`, что привело бы к ошибке при компиляции. Поскольку некоторые распределения вызывают процессор несколько раз, передается сам процессор, а не очередной результат его вызова.



Когда упоминается *генератор случайных чисел* (random-number generator), имеется в виду комбинация объекта распределения с объектом процессора.

Сравнение процессора случайных чисел и функции `rand()`

Читатели, знакомые с библиотечной функцией `rand()` языка C, вероятно заметили, что вывод вызова объекта `default_random_engine` подобен выводу функции `rand()`. Процессоры предоставляют целые беззнаковые числа в определенном системой диапазоне. Функция `rand()` имеет диапазон от 0 до `RAND_MAX`. Диапазон процессора возвращается при вызове функций-членов `min()` и `max()` объекта его типа:

```
cout << "min: " << e.min() << " max: " << e.max() << endl;
```

На системе авторов эта программа выводит следующее:

```
min: 1 max: 2147483646
```

Таблица 17.15. Операции с процессором случайного числа

<code>Engine e;</code>	Стандартный конструктор; использует заданное по умолчанию начальное число для типа процессора
<code>Engine e(s);</code>	Использует как начальное число целочисленное значение <code>s</code>
<code>e.seed(s)</code>	Переустанавливает состояние процессора, используя начальное число <code>s</code>
<code>e.min()</code> <code>e.max()</code>	Наименьшие и наибольшие числа, создаваемые данным генератором
<code>Engine::result_type</code>	Целочисленный беззнаковый тип, создаваемый данным процессором
<code>e.discard(u)</code>	Перемещает процессор на <code>u</code> шагов; <code>u</code> имеет тип <code>unsigned long long</code>

Процессоры создают последовательности чисел

У генераторов случайных чисел есть одно свойство, которое зачастую вызывает сомнения у новичков: даже при том, что создаваемые числа случайны, при каждом запуске данный генератор возвращает ту же последовательность чисел. Факт неизменности последовательности очень полезен во время проверки. С другой стороны, разработчики, использующие генераторы случайных чисел, должны учитывать этот факт.

Предположим, например, что необходима функция, создающая вектор из 100 случайных целых чисел, равномерно распределенных в диапазоне от 0 до 9. Могло бы показаться, что эту функцию следует написать следующим образом:

```
// безусловно неправильный способ создания вектора случайных целых чисел
// эта функция выводит те же 100 чисел при каждом вызове!
vector<unsigned> bad_randVec()
{
    default_random_engine e;
    uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

Однако при каждом вызове эта функция возвратит тот же вектор:

```
vector<unsigned> v1(bad_randVec());
vector<unsigned> v2(bad_randVec());
```

```
// выводит equal
cout << ((v1 == v2) ? "equal" : "not equal") << endl;
```

Этот код выводит "equal", поскольку векторы `v1` и `v2` имеют те же значения.

Для правильного написания этой функции объекты процессора и распределения следует сделать статическими (см. раздел 6.1.1, стр. 272):

```
// возвращает вектор из 100 равномерно распределенных случайных чисел
vector<unsigned> good_randVec()
{
    // поскольку процессоры и распределения хранят состояние, их следует
    // сделать статическими, чтобы при каждом вызове создавались новые
    // числа
    static default_random_engine e;
    static uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

Поскольку объекты `e` и `u` являются статическими, они хранят свое состояние на протяжении вызовов функции. Первый вызов будет использовать первые 100 случайных чисел из последовательности, созданной вызовом `u(e)`, а второй вызов создаст следующие 100 чисел и т.д.



Каждый генератор случайных чисел всегда создает ту же последовательность чисел. Функция с локальным генератором случайных чисел должна сделать объекты процессора и распределения статическими. В противном случае функция будет создавать ту же последовательность при каждом вызове.

Начальное число генератора

Тот факт, что генератор возвращает ту же последовательность чисел, полезен во время отладки. Но после проверки программы необходимо заставить ее создавать разные случайные результаты при каждом запуске. Для этого предоставляется *начальное число* (seed). Начальное число — это значение, которое процессор может использовать для начала создания чисел с нового пункта в последовательности.

Начальное число генератора можно задать одним из двух способов: предоставить его при создании объекта процессора либо вызвать функцию-член `seed()` класса процессора:

```
default_random_engine e1; // использует стандартное начальное число
default_random_engine e2(2147483646); // использует заданное значение
// начального числа
// e3 и e4 создадут ту же последовательность, поскольку они используют то
```

```
// же начальное число
default_random_engine e3; // использует стандартное начальное число
e3.seed(32767); // вызывает функцию seed() для установки нового значения
// начального числа
default_random_engine e4(32767); // устанавливает начальное число 32767
for (size_t i = 0; i != 100; ++i) {
    if (e1() == e2())
        cout << "unseeded match at iteration: " << i << endl;
    if (e3() != e4())
        cout << "seeded differs at iteration: " << i << endl;
}
```

Здесь определены четыре процессора. Первые два, `e1` и `e2`, имеют разные начальные числа и *должны* создавать разные последовательности. У двух вторых, `e3` и `e4`, то же значение начального числа. Эти два объекта создадут ту же последовательность.

Выбор подходящего начального числа, как и почти все при создании хороших наборов случайных чисел, на удивление сложен. Вероятно, наиболее распространен подход вызова системной функции `time()`. Эта функция, определенная в заголовке `ctime`, возвращает количество секунд, начиная с заданной эпохи. Функция `time()` получает один параметр, являющийся указателем на структуру для записи времени. Если этот указатель нулевой, функция только возвращает время:

```
default_random_engine e1(time(0)); // почти случайное начальное число
```

Поскольку функция `time()` возвращает время как количество секунд, такое начальное число применимо только для приложений, создающих начальное число на уровне секунд или больших интервалов.



ВНИМАНИЕ

Функция `time()` обычно не используется как источник начального числа, если программа многократно запускается как часть автоматизированного процесса, поскольку она могла бы быть запущена с тем же начальным числом несколько раз.

Упражнения раздела 17.4.1

Упражнение 17.28. Напишите функцию, создающую и возвращающую равномерно распределенную последовательность случайных беззнаковых целых чисел при каждом вызове.

Упражнение 17.29. Позвольте пользователю предоставлять начальное число как необязательный аргумент функции, написанной в предыдущем упражнении.

Упражнение 17.30. Снова пересмотрите предыдущую функцию, позволив ей получать минимальное и максимальное значения для возвращаемых случайных чисел.

17.4.2. Другие виды распределений

Процессоры создают беззнаковые числа, и у каждого числа в диапазоне процессора есть та же вероятность быть созданным. Приложения зачастую нуждаются в числах других типов или распределений. Библиотека удовлетворяет обе эти потребности, определяя различные классы распределений, которые, будучи использованы с процессором, дают желаемый результат. Список операций, поддерживаемых типами распределения, приведен в табл. 17.16.

Таблица 17.16. Операции с распределениями

<i>Dist d;</i>	Стандартный конструктор; создает объект <i>d</i> готовым к использованию. Другие конструкторы зависят от типа <i>Dist</i> ; см. раздел A.3 (стр. 1098). Конструкторы распределений являются явными (см. раздел 7.5.4, стр. 383)
<i>d(e)</i>	Последовательные вызовы с тем же объектом <i>e</i> создадут последовательность случайных чисел согласно типу распределения <i>d</i> ; <i>e</i> — объект процессора случайных чисел
<i>d.min()</i> <i>d.max()</i>	Возвращает наименьшее и наибольшее числа, создаваемые <i>d(e)</i>
<i>d.reset()</i>	Восстанавливает состояние объекта <i>d</i> , чтобы последующее его использование не зависело от уже созданных значений

Создание случайных вещественных чисел

Программы нередко нуждаются в источнике случайных значений с плавающей точкой. В частности, в диапазоне от нуля до единицы.

Наиболее распространен *неправильный* способ получения случайного числа с плавающей точкой из функции `rand()` за счет деления результата ее выполнения на значение `RAND_MAX`, являющееся заданным системой верхним пределом случайного числа, возвращаемого функцией `rand()`. Этот подход *неправильный* потому, что у случайных целых чисел обычно меньшая точность, чем у чисел с плавающей запятой, поэтому некоторые значения с плавающей точкой никогда не будут получены.

Новые библиотечные средства позволяют легко получить случайное число с плавающей точкой. Достаточно определить объект типа `uniform_real_distribution` и позволить библиотеке соотнести случайные целые числа с числам с плавающей запятой. Подобно типу `uniform_int_distribution`, здесь также можно задать минимальные и максимальные значения при определении объекта:

```
default_random_engine e; // создает случайные беззнаковые целые числа
// однородное распределение от 0 до 1 включительно
uniform_real_distribution<double> u(0,1);
```

```
for (size_t i = 0; i < 10; ++i)
    cout << u(e) << " ";
```

Этот код почти идентичен предыдущей программе, которая создавала беззнаковые значения. Но поскольку здесь использован другой тип распределения, данная версия дает другие результаты:

```
0.131538 0.45865 0.218959 0.678865 0.934693 0.519416 ...
```

Использование типа по умолчанию для результата распределения

За одним исключением, рассматриваемым в разделе 17.4.2 (стр. 941), типы распределения являются шаблонами с одним параметром типа шаблона, представляющим тип создаваемых распределением чисел. Эти типы всегда создают либо тип с плавающей точкой, либо целочисленный тип.

У каждого шаблона распределения есть аргумент шаблона по умолчанию (см. раздел 16.1.3, стр. 844). Типы распределения, создающие значения с плавающей точкой, по умолчанию создают значения типа `double`. Распределения, создающие целочисленные результаты, используют по умолчанию тип `int`. Поскольку у типов распределения есть только один параметр шаблона, при необходимости использовать значение по умолчанию следует не забыть расположить за именем шаблона пустые угловые скобки, чтобы указать на применение типа по умолчанию (см. раздел 16.1.3, стр. 845):

```
// пустые <> указывают на использование для результата типа по умолчанию
uniform_real_distribution<> u(0,1); // по умолчанию double
```

Создание чисел с неравномерным распределением

Кроме корректного создания случайных чисел в заданном диапазоне, новая библиотека позволяет также получить числа, распределенные неравномерно. Действительно, библиотека определяет 20 типов распределений! Эти типы перечисляются в разделе A.3 (стр. 1098).

Для примера создадим серию нормально распределенных значений и нарисуем полученное распределение. Поскольку тип `normal_distribution` создает числа с плавающей запятой, данная программа будет использовать функцию `lround()` из заголовка `cmath` для округления каждого результата до ближайшего целого числа. Создадим 200 чисел с центром в значении 4 и среднеквадратичным отклонением 1.5. Поскольку используется нормальное распределение, можно ожидать любых чисел, но приблизительно 1% из них будет в диапазоне от 0 до 8 включительно. Программа подсчитывает, сколько значений соответствует каждому целому числу в этом диапазоне:

```
default_random_engine e;           // создает случайные целые числа
normal_distribution<> n(4,1.5);     // середина 4, среднеквадратичное
                                   // отклонение 1.5
```



```
vector<unsigned> vals(9);           // девять элементов со значением 0
for (size_t i = 0; i != 200; ++i) {
    unsigned v = lround(n(e));     // округление до ближайшего целого
    if (v < vals.size())           // если результат в диапазоне
        ++vals[v]; // подсчитать, как часто встречается каждое число
}
for (size_t j = 0; j != vals.size(); ++j)
    cout << j << ": " << string(vals[j], '*') << endl;
```

Начнем с определения объектов генератора случайных чисел и вектора `vals`. Вектор `vals` будет использован для расчета частоты создания каждого числа в диапазоне 0...9. В отличие от большинства других программ, использующих вектор, создадим его сразу с необходимым размером. Так, каждый его элемент инициализируется значением 0.

В цикле `for` происходит вызов функции `lround(n(e))` для округления возвращенного вызовом `n(e)` значения до ближайшего целого числа. Получив целое число, соответствующее случайному числу с плавающей точкой, используем его для индексирования вектора счетчиков. Поскольку вызов `n(e)` может создавать числа и вне диапазона от 0 до 9, проверим полученное число на принадлежность диапазону прежде, чем использовать его для индексирования вектора `vals`. Если число принадлежит диапазону, увеличиваем соответствующий счетчик.

Когда цикл заканчивается, вывод содержимого вектора `vals` выглядит следующим образом:

```
0: ***
1: *****
2: *****
3: *****
4: *****
5: *****
6: *****
7: *****
8: *
```

Выведенные строки содержат столько звездочек, сколько раз встретилось соответствующее значение, созданное генератором случайных чисел. Обратите внимание: эта фигура не совершенно симметрична. Если бы она была симметрична, то возникли бы подозрения в качестве генератора случайных чисел.

Класс `bernoulli_distribution`

Как уже упоминалось, есть одно распределение, которое не получает параметр шаблона. Это распределение `bernoulli_distribution`, являющееся обычным классом, а не шаблоном. Это распределение всегда возвращает логическое значение `true` с заданной вероятностью. По умолчанию это вероятность .5.

В качестве примера распределения этого вида напомним программу, которая играет с пользователем. Игру начинает один из игроков (пользователь или программа). Чтобы выбрать первого игрока, можно использовать объект класса `uniform_int_distribution` с диапазоном от 0 до 1. В качестве альтернативы этот выбор можно сделать, используя распределение Бернулли. С учетом, что игру начинает функция `play()`, для взаимодействия с пользователем может быть использован следующий цикл:

```
string resp;
default_random_engine e; // e имеет состояние, поэтому располагается
                          // вне цикла!
bernoulli_distribution b; // по умолчанию четность 50/50
do {
    bool first = b(e); // если true, программа ходит первой
    cout << (first ? "We go first"
                : "You get to go first") << endl;
    // играть в игру, угадывая, кто ходит первым
    cout << ((play(first)) ? "sorry, you lost"
                : "congrats, you won") << endl;
    cout << "play again? Enter 'yes' or 'no'" << endl;
} while (cin >> resp && resp[0] == 'y');
```

Для повторного запроса на продолжение игры используем цикл `do while` (см. раздел 5.4.4, стр. 253).



ВНИМАНИЕ

Поскольку процессоры возвращают ту же последовательность чисел (см. раздел 17.4.1, стр. 936), их объявляют за пределами циклов. В противном случае при каждой итерации создавался бы новый процессор, выдающий каждый раз те же значения. Распределения также могут хранить состояние и также должны определяться вне циклов.

Одна из причин использования в этой программе распределения `bernoulli_distribution` заключается в том, что это предоставит программе лучший шанс пойти первой:

```
bernoulli_distribution b(.55); // предоставить программе небольшое
                               // преимущество
```

Такое определение `b` предоставит программе 55/45 шансов на первый ход.

Упражнения раздела 17.4.2

Упражнение 17.31. Что случилось бы в программе игры данного раздела, будь объекты `b` и `e` определены в цикле `do`?

Упражнение 17.32. Что случилось бы, будь строка `resp` определена в цикле?

Упражнение 17.33. Напишите версию программы преобразования слова из раздела 11.3.6 (стр. 562), допускающую несколько преобразований для заданного слова и случайно выбирающую применяемое преобразование.

17.5. Еще о библиотеке ввода и вывода

Глава 8 познакомила вас с базовой архитектурой и наиболее часто используемой частью библиотеки ввода-вывода. В этом разделе рассматриваются три более специализированных средства, поддерживаемых библиотекой ввода-вывода: управление форматом, не форматированный ввод-вывод и произвольный доступ.

17.5.1. Форматированный ввод и вывод

Кроме флага состояния (см. раздел 8.1.2, стр. 402), каждый объект `istream` имеет также флаг формата, контролирующий подробности формата ввода и вывода. Флаг формата контролирует такие аспекты, как формат записи целочисленных значений, точность значений с плавающей запятой, ширина выводимого элемента и т.д.

Библиотека определяет набор перечисленных в табл. 17.17 и 17.18 *манипуляторов* (*manipulator*) (см. раздел 1.2, стр. 30), изменяющих флаг формата потока. Манипулятор — это функция или объект, влияющие на состояние потока и применяемые как операнд оператора ввода или вывода. Как и операторы ввода и вывода, манипулятор возвращает потоковый объект, к которому он применяется; таким образом, можно объединить манипуляторы и данные в один оператор.

Таблица 17.17. Манипуляторы, определенные в объекте `istream`

<code>boolalpha</code>	Отображать значения <code>true</code> и <code>false</code> как строки
<code>* noboolalpha</code>	Отображать значения <code>true</code> и <code>false</code> как 0 и 1
<code>showbase</code>	Создавать префикс, означающий базу целочисленных значений
<code>* noshowbase</code>	Не создавать префикс базы чисел
<code>showpoint</code>	Всегда отображать десятичную точку для значений с плавающей запятой
<code>* noshowpoint</code>	Отображать десятичную точку, только если у значения есть дробная часть
<code>showpos</code>	Отображать + для положительных чисел
<code>* noshowpos</code>	Не отображать + в неотрицательных числах
<code>uppercase</code>	Выводить 0x в шестнадцатеричной и E в экспоненциальной формах записи
<code>* nouppercase</code>	Выводить 0x в шестнадцатеричной и e в экспоненциальной формах записи
<code>* dec</code>	Отображать целочисленные значения с десятичной базой числа
<code>hex</code>	Отображать целочисленные значения с шестнадцатеричной базой числа

Окончание табл. 17.17

<code>oct</code>	Отображать целочисленные значения с восьмеричной базой числа
<code>left</code>	Добавлять дополняющие символы справа от значения
<code>right</code>	Добавлять дополняющие символы слева от значения
<code>internal</code>	Добавлять дополняющие символы между знаком и значением
<code>fixed</code>	Отображать значения с плавающей точкой в десятичном представлении
<code>scientific</code>	Отображать значения с плавающей точкой в экспоненциальном представлении
<code>hexfloat</code>	Отображать значения с плавающей точкой в шестнадцатеричном представлении (нововведение C++11)
<code>defaultfloat</code>	Вернуть формат числа с плавающей точкой в десятичный (нововведение C++11)
<code>unitbuf</code>	Сбрасывать буфер после каждой операции вывода
<code>·nobuf</code>	Восстановить обычный сброс буфера
<code>·skipws</code>	Пропускать отступы в операторах ввода
<code>noskipws</code>	Не пропускать отступы в операторах ввода
<code>flush</code>	Сбросить буфер объекта <code>ostream</code>
<code>ends</code>	Вставить нулевой символ, а затем сбросить буфер объекта <code>ostream</code>
<code>endl</code>	Вставить новую строку, а затем сбросить буфер объекта <code>ostream</code>

· Означает стандартное состояние потока

Таблица 17.18. Манипуляторы, определенные в объекте `iomanip`

<code>setfill(ch)</code>	Заполнить отступ символом <code>ch</code>
<code>setprecision(n)</code>	Установить точность <code>n</code> числа с плавающей точкой
<code>setw(w)</code>	Читать или писать значение в <code>w</code> символов
<code>setbase(b)</code>	Вывод целых чисел с базой <code>b</code>

Ранее в программах уже использовался манипулятор `endl`, который “записывался” в поток вывода как будто это значение. Но манипулятор `endl` — не обычное значение; он выполняет операцию: выводит символ новой строки и сбрасывает буфер.

Большинство манипуляторов изменяет флаг формата

Манипуляторы используются для двух общих категорий управления выводом: контроль представления числовых значений, а также контроль количества и расположения заполнителей. Большинство манипуляторов, изменяющих флаг формата, предоставлены парами для установки и сброса; один манипуля-

тор устанавливает флаг формата в новое значение, а другой сбрасывает его, восстанавливая стандартное значение.



ВНИМАНИЕ

Манипуляторы, изменяющие флаг формата потока, обычно оставляют флаг формата измененным для всего последующего ввода-вывода.

Тот факт, что манипулятор вносит постоянное изменение во флаг формата, может оказаться полезным, когда имеется ряд операций ввода-вывода, использующих одинаковое форматирование. Действительно, некоторые программы используют эту особенность манипуляторов для изменения поведения одного или нескольких правил форматирования ввода или вывода. В таких случаях факт изменения потока является желательным.

Но большинство программ (и что еще важнее, разработчиков) ожидают, что состояние потока будет соответствовать стандартным библиотечным значениям. В этих случаях оставленный в нестандартном состоянии поток может привести к ошибке. В результате обычно лучше отменить изменение состояния, как только оно больше не нужно.

Контроль формата логических значений

Хорошим примером манипулятора, изменяющего состояние формата своего объекта, является манипулятор `boolalpha`. По умолчанию значение типа `bool` выводится как 1 или 0. Значение `true` выводится как целое число 1, а значение `false` как 0. Это поведение можно переопределить, применив к потоку манипулятор `boolalpha`:

```
cout << "default bool values: " << true << " " << false
    << "\nalpha bool values: " << boolalpha
    << true << " " << false << endl;
```

Эта программа выводит следующее:

```
default bool values: 1 0
alpha bool values: true false
```

Как только манипулятор `boolalpha` “записан” в поток `cout`, способ вывода логических значений изменяется. Последующие операции вывода логических значений отобразят их как “true” или “false”.

Чтобы отменить изменение флага формата потока `cout`, применяется манипулятор `noboolalpha`:

```
bool bool_val = get_status();
cout << boolalpha      // устанавливает внутреннее состояние cout
    << bool_val
    << noboolalpha;    // возвращает стандартное внутреннее состояние
```

Здесь формат вывода логических значений изменен только для вывода значения `bool_val`. Как только это значение будет выведено, поток немедленно возвращается в первоначальное состояние.

Определение базы целочисленных значений

По умолчанию целочисленные значения выводятся и читаются в десятичном формате. Используя манипуляторы `hex`, `oct` и `dec`, базу записи числа можно изменить на восьмеричную, шестнадцатеричную и обратно на десятичную базу:

```
cout << "default: " << 20 << " " << 1024 << endl;
cout << "octal: " << oct << 20 << " " << 1024 << endl;
cout << "hex: " << hex << 20 << " " << 1024 << endl;
cout << "decimal: " << dec << 20 << " " << 1024 << endl;
```

После компиляции и запуска на выполнение эта программа выводит следующее:

```
default: 20 1024
octal: 24 2000
hex: 14 400
decimal: 20 1024
```

Обратите внимание, как и манипулятор `boolalpha`, эти манипуляторы изменяют флаг формата. Они срабатывают сразу после применения и влияют на весь последующий вывод целочисленных значений, пока формат не изменит применение другого манипулятора.



Манипуляторы `hex`, `oct` и `dec` влияют на вывод только целочисленных операндов, но не значений с плавающей запятой.

Индикация базы числа в выводе

По умолчанию при выводе числа нет никакого визуального уведомления об используемой базе. Например, 20 — это действительно 20, или восьмеричное представление числа 16? Когда числа выводятся в десятичном режиме, они отображаются, как и ожидается. Если необходимо вывести восьмеричные или шестнадцатеричные значения, вероятней всего, придется использовать также манипулятор `showbase`. Он заставляет поток вывода использовать те же соглашения, что и при определении базы целочисленных констант.

- Предваряющий `0x` означает шестнадцатеричный формат.
- Предваряющий `0` означает восьмеричный формат.
- Отсутствие любого индикатора означает десятичное число.

Здесь предыдущая программа пересмотрена для использования манипулятора `showbase`:

```
cout << showbase; // отображать базу при выводе целочисленных значений
cout << "default: " << 20 << " " << 1024 << endl;
cout << "in octal: " << oct << 20 << " " << 1024 << endl;
cout << "in hex: " << hex << 20 << " " << 1024 << endl;
cout << "in decimal: " << dec << 20 << " " << 1024 << endl;
cout << noshowbase; // вернуть состояние потока
```

Вывод пересмотренной программы проясняет смысл:

```
default: 20 1024
in octal: 024 02000
in hex: 0x14 0x400
in decimal: 20 1024
```

Манипулятор `noshowbase` возвращает поток `cout` в прежнее состояние, когда индикатор базы не отображается.

По умолчанию шестнадцатеричные значения выводятся в нижнем регистре `x`, также в нижнем регистре. Манипулятор `uppercase` позволяет отобразить `X` и шестнадцатеричные цифры `a–f` в верхнем регистре:

```
cout << uppercase << showbase << hex
    << "printed in hexadecimal: " << 20 << " " << 1024
    << nouppercase << noshowbase << dec << endl;
```

Этот оператор создает следующий вывод:

```
printed in hexadecimal: 0X14 0X400
```

Манипуляторы `nouppercase`, `noshowbase` и `dec` применяются для возвращения потока в исходное состояние.

Контроль формата значений с плавающей точкой

Контролировать можно три аспекта вывода числа с плавающей запятой.

- Количество выводимых цифр точности.
- Выводится ли число в шестнадцатеричном формате, как фиксированное десятичное число или в экспоненциальном представлении.
- Выводится ли десятичная точка для целочисленных значений с плавающей запятой.

По умолчанию значения с плавающей запятой выводятся с шестью цифрами точности; десятичная точка не отображается при отсутствии дробной части; в зависимости от величины значения используется фиксированный десятичный формат или экспоненциальная форма. Библиотека выбирает формат, увеличивающий удобочитаемость числа. Очень большие и очень маленькие значения выводятся в экспоненциальном представлении. Другие значения выводятся в фиксированном десятичном формате.

Определение точности

По умолчанию точность контролирует общее количество отображаемых цифр. При выводе значение с плавающей запятой округляется (а не усекается) до текущей точности. Таким образом, если текущая точность четыре, то число 3.14159 становится 3.142; если точность три, то оно выводится как 3.14.

Для изменения точности можно воспользоваться функцией-членом `precision()` объекта ввода-вывода или манипулятором `setprecision`. Функция-член `precision()` перегружена (см. раздел 6.4, стр. 302). Одна ее версия получает значение типа `int` и устанавливает точность в это новое значение. Она возвращает *предыдущее* значение точности. Другая версия не получает никаких аргументов и возвращает текущее значение точности. Манипулятор `setprecision` получает аргумент, который и использует для установки точности.



Манипулятор `setprecision` и другие манипуляторы, получающие аргументы, определяются в заголовке `iomanip`.

Следующая программа иллюстрирует различные способы контроля точности при выводе значения с плавающей точкой:

```
// cout.precision() сообщает текущее значение точности
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
// cout.precision(12) запрашивает вывод 12 цифр точности
cout.precision(12);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
// альтернативный способ установки точности с использованием
// манипулятора setprecision
cout << setprecision(3);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
```

Эта программа выводит следующее:

```
Precision: 6, Value: 1.41421
Precision: 12, Value: 1.41421356237
Precision: 3, Value: 1.41
```

Программа использует библиотечную функцию `sqrt()`, определенную в заголовке `cmath`. Функция `sqrt()` перегружена и может быть вызвана с аргументами типа `float`, `double` или `long double`. Она возвращает квадратный корень своего аргумента.

Определение формы записи чисел с плавающей запятой



Если нет реальной необходимости контролировать представление числа с плавающей запятой (например, для вывода данных в столбик, отображения денежных данных или процентов), лучше позволить библиотеке выбирать форму записи самостоятельно.

Используя соответствующий манипулятор, можно заставить поток использовать научную, фиксированную или шестнадцатеричную форму записи. Манипулятор `scientific` задает использование экспоненциального представления. Манипулятор `fixed` задает использование фиксированных десятичных чисел.

**C++
11**

Новая библиотека позволяет выводить значения с плавающей точкой в шестнадцатеричном формате при помощи манипулятора `hexfloat`. Новая библиотека предоставляет еще один манипулятор, `defaultfloat`. Он возвращает поток в стандартное состояние, при котором выбор формы записи осуществляется на основании выводимого значения.

Эти манипуляторы изменяют также заданное для потока по умолчанию значение точности. После применения манипуляторов `scientific`, `fixed` или `hexfloat` значение точности контролирует количество цифр после десятичной точки. По умолчанию точность определяет количество цифр до и после десятичной точки. Манипуляторы `fixed` и `scientific` позволяют выводить числа, выстроенные в столбцы, с десятичной точкой в фиксированной позиции относительно дробной части:

```
cout << "default format: " << 100 * sqrt(2.0) << '\n'
    << "scientific: " << scientific << 100 * sqrt(2.0) << '\n'
    << "fixed decimal: " << fixed << 100 * sqrt(2.0) << '\n'
    << "hexadecimal: " << hexfloat << 100 * sqrt(2.0) << '\n'
    << "use defaults: " << defaultfloat << 100 * sqrt(2.0)
    << "\n\n";
```

Получается следующий вывод:

```
default format: 141.421
scientific: 1.414214e+002
fixed decimal: 141.421356
hexadecimal: 0x1.lad7bcp+7
use defaults: 141.421
```

По умолчанию шестнадцатеричные цифры и символ `e`, используемый в экспоненциальном представлении, выводятся в нижнем регистре. Манипулятор `uppercase` позволяет выводить эти значения в верхнем регистре.

Вывод десятичной точки

По умолчанию, когда дробная часть значения с плавающей точкой равна 0, десятичная точка не отображается. Манипулятор `showpoint` требует отображать десятичную точку всегда:

```
cout << 10.0 << endl;           // выводит 10
cout << showpoint << 10.0       // выводит 10.0000
    << noshowpoint << endl;     // возвращает стандартный формат десятичной
                                // точки
```

Манипулятор `noshowpoint` восстанавливает стандартное поведение. У вывода следующих выражений будет стандартное поведение, подразумевающее отсутствие десятичной точки, если дробная часть значения с плавающей точкой отсутствует.

Дополнение вывода

При выводе данных в столбцах зачастую необходим довольно подробный контроль над форматированием данных. Библиотека предоставляет несколько манипуляторов, обеспечивающих контроль, который может понадобиться.

- Манипулятор `setw` задает минимальное пространство для *следующего* числового или строкового значения.
- Манипулятор `left` выравнивает текст по левому краю вывода.
- Манипулятор `right` выравнивает текст по правому краю (принято по умолчанию).
- Манипулятор `internal` контролирует положение знака отрицательных значений. Выравнивает знак по левому краю, а значение по правому, дополняя пространство между ними пробелами.
- Манипулятор `setfill` позволяет задать альтернативный символ для дополнения вывода. По умолчанию принят пробел.



Манипуляторы `setw` и `endl` не изменяют внутреннее состояние потока вывода. Они определяют только *последующий* вывод.

Эти манипуляторы иллюстрирует следующая программа:

```
int i = -16;
double d = 3.14159;
// дополняет первый столбец, обеспечивая минимум 12 позиций вывода
cout << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// дополняет первый столбец и выравнивает все столбцы по левому краю
cout << left
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n'
    << right; // восстанавливает стандартное выравнивание
// дополняет первый столбец и выравнивают все столбцы по правому краю
cout << right
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// дополняет первый столбец и помещает дополнение в поле
cout << internal
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// дополняет первый столбец, используя символ # как заполнитель
cout << setfill('#')
    << "i: " << setw(12) << i << "next col" << '\n'
```

```
<< "d: " << setw(12) << d << "next col" << '\n'
<< setfill(' '); // восстанавливает стандартный символ заполнения
```

Вывод этой программы таков:

```
i:          -16next col
d:          3.14159next col
i: -16              next col
d: 3.14159          next col
i:          -16next col
d:          3.14159next col
i: -              16next col
d:          3.14159next col
i: -#####16next col
d: #####3.14159next col
```

Контроль формата ввода

По умолчанию операторы ввода игнорируют символы отступа (пробел, табуляция, новая строка, новая страница и возврат каретки).

```
char ch;
while (cin >> ch)
    cout << ch;
```

Этот цикл получает следующую исходную последовательность:

```
a b    c
d
```

Он выполняется четыре раза, читая символы от a до d, пропуская промежуточные пробелы, возможные символы табуляции и новой строки. Вывод этой программы таков:

```
abcd
```

Манипулятор `noskipws` заставляет оператор ввода читать, не игнорируя отступ. Для возвращения к стандартному поведению применяется манипулятор `skipws`:

```
cin >> noskipws; // установить cin на чтение отступа
while (cin >> ch)
    cout << ch;
cin >> skipws; // вернуть cin к стандартному игнорированию отступа
```

При том же вводе этот цикл делает семь итераций, читая отступы как символы во вводе. Его вывод таков:

```
a b    c
d
```

Упражнения раздела 17.5.1

Упражнение 17.34. Напишите программу, иллюстрирующую использование каждого манипулятора из табл. 17.17 и 17.18 (стр. 944).

Упражнение 17.35. Напишите версию программы вывода квадратного корня со стр. 948, но выводящую на сей раз шестнадцатеричные цифры в верхнем регистре.

Упражнение 17.36. Измените программу из предыдущего упражнения так, чтобы различные значения с плавающей точкой выводились в столбце.

17.5.2. Не форматированные операции ввода-вывода

До сих пор в программах использовались только операции *форматированного ввода-вывода* (formatted IO). Операторы ввода и вывода (<< и >>) форматировуют читаемые и выводимые данные согласно их типу. Операторы ввода игнорируют отступ; операторы вывода применяют дополнение, точность и т.д.

Библиотека предоставляет также набор низкоуровневых функций *не форматированного ввода-вывода* (unformatted IO). Эти функции позволяют работать с потоком как с последовательностью неинтерпретируемых байтов.

Однобайтовые операции

Некоторые из не форматированных операций имеют дело с обработкой потока по одному байту за раз. Они описаны в табл. 17.19 и читают данные, не игнорируя отступ. Например, функции не форматированного ввода-вывода `get()` и `put()` позволяют читать и записывать символы по одному:

```
char ch;
while (cin.get(ch))
    cout.put(ch);
```

Эта программа сохраняет отступ во вводе. Ее вывод идентичен вводу. Она работает так же, как и предыдущая программа, использовавшая манипулятор `noskipws`.

Таблица 17.19. Однобайтовые низкоуровневые функции ввода-вывода

<code>is.get(ch)</code>	Помещает следующий байт из потока <code>is</code> класса <code>istream</code> в символьную переменную <code>ch</code> . Возвращает поток <code>is</code>
<code>os.put(ch)</code>	Помещает символ <code>ch</code> в поток <code>os</code> класса <code>ostream</code> . Возвращает поток <code>os</code>
<code>is.get()</code>	Возвращает следующий байт из потока <code>is</code> как тип <code>int</code>
<code>is.putback(ch)</code>	Помещает символ <code>ch</code> назад в поток <code>is</code> ; возвращает поток <code>is</code>
<code>is.unget()</code>	Перемещает в поток <code>is</code> один байт; возвращает поток <code>is</code>
<code>is.peek()</code>	Возвращает следующий байт как тип <code>int</code> , но не удаляет его

Возвращение во входной поток

Иногда необходимо читать отдельные символы так, чтобы знать, к чему быть готовым. В таких случаях символы желательно возвращать в поток. Библиотека предоставляет три способа сделать это, и у каждого из них есть свои отличия.

- Функция `peek()` возвращает копию следующего символа во входном потоке, но не изменяет поток. Возвращенное значение остается в потоке.
- Функция `unget()` создает резервную копию входного потока, чтобы независимо от того, какое значение было последним возвращенным, оно все еще оставалось в потоке. Функцию `unget()` можно вызвать, даже не зная, какое значение было извлечено из потока последним.
- Функция `putback()` — это более специализированная версия функции `unget()`: она возвращает последнее прочитанное из потока значение, но получает аргумент, который должен совпадать с последним прочитанным значением.

Таким образом, они гарантируют возможность вернуть в поток как минимум одно значение перед следующим чтением. Следовательно, гарантированно не получится вызвать функции `putback()` или `unget()` последовательно, без промежуточной операции чтения.

Возвращение значения типа `int` из операций ввода

Функция `peek()` и версия функции `get()` без аргументов возвращают прочитанный символ из входного потока как значение типа `int`. Этот факт может удивить; казалось бы, более естественным было бы возвращение типа `char`.

Причина возвращения этими функциями типа `int` в том, чтобы позволить им возвратить маркер конца файла. Полученный набор символов позволяет использовать каждое значение в диапазоне типа `char` и представлять фактические символы. Но в этом диапазоне нет никакого специального значения для представления конца файла.

Функции, возвращающие тип `int`, преобразуют возвращаемый символ в тип `unsigned char`, а затем преобразуют это значение в тип `int`. В результате, даже если в наборе символов будут символы, соответствующие отрицательным значениям, возвращенный этими функциями тип `int` будет иметь положительное значение (см. раздел 2.1.2, стр. 66). Библиотека использует отрицательное значение для представления конца файла, гарантируя таким образом его отличие от любого настоящего символьного значения. Чтобы не обязывать разработчиков знать фактическое возвращаемое значение, заголовок `iostream` определяет константу `EOF`, которую можно использовать для проверки, не является ли возвращенное функцией `get()` значение концом

файла. Вот почему для содержания значения, возвращаемого этими функциями, используется переменная типа `int`.

```
int ch; // возвращаемое fromget() значение содержится в int, а не char
// цикл чтения и записи всех данных во вводе
while ((ch = cin.get()) != EOF)
    cout.put(ch);
```

Эта программа работает так же, как и прежняя, но здесь для чтения ввода используется функция `get()`.

ВНИМАНИЕ! НИЗКОУРОВНЕВЫЕ ФУНКЦИИ ПОДВЕРЖЕНЫ ОШИБКАМ

Обычно рекомендуется использовать высокоуровневые абстракции, предоставляемые библиотекой. Функции ввода-вывода, возвращающие значение типа `int`, являются хорошим подтверждением правильности этой рекомендации.

Обычной ошибкой программирования является присвоение значения, возвращаемого функцией `get()` или `peek()`, возвращающей тип `int`, переменной типа `char`, а не `int`. Это, безусловно, будет ошибкой, но компилятор ее не обнаружит. То, что произойдет в результате этой ошибки, зависит от конкретной машины и введенных данных. Например, если машина интерпретирует символ как беззнаковое целое число, приведенный ниже цикл окажется бесконечным.

```
char ch; // применение типа char здесь приведет к катастрофе!
// значение, возвращенное функцией get() объекта cin,
// преобразуется из int в char, а затем сравнивается с int
while ((ch = cin.get()) != EOF)
    cout.put(ch);
```

Проблема в том, что когда функция `get()` возвращает значение `EOF`, оно преобразуется в беззнаковое значение типа `unsigned char`. Это преобразованное значение не будет равно целочисленному значению `EOF`, поэтому цикл не закончится никогда. Такие ошибки обычно обнаруживаются при проверке.

Но нельзя быть уверенным в том, что на тех машинах, где символы интерпретируются как знаковый тип, поведение цикла будет аналогичным. Ведь результат переполнения переменной беззнакового типа зависит от компилятора. На большинстве машин этот цикл будет работать нормально, если только во вводимых данных не встретится символ, соответствующий значению `EOF`. Поскольку в обычных данных такие символы маловероятны, низкоуровневые операторы ввода-вывода могут пригодиться при чтении только бинарных значений, которые не соответствуют непосредственно обычным символам и числовым значениям. На машине автора, например, цикл преждевременно завершается в случае ввода символа, значением которого является `'\377'`. Когда значение `'\377'` на машине автора преобразуется в тип `signed char`, полу-

чается значение `-1`. Если во введенных данных встретится это значение, оно будет рассматриваться как символ (преждевременного) конца файла.

При чтении и записи типизированных значений такие ошибки не возникают. Поэтому по возможности следует использовать предоставляемые библиотекой высокоуровневые операторы, что гораздо безопасней.

Многобайтовые операции

Некоторые операции не форматированного ввода-вывода работают с порциями данных за раз. Эти операции могут быть полезны, если важна скорость, но, как и другие низкоуровневые операции, они подвержены ошибкам. В частности эти операции требуют резервирования и управления символьными массивами (см. раздел 12.2, стр. 606), используемыми для сохранения и вращения данных. Многобайтовые операции перечислены в табл. 17.20.

Таблица 17.20. Многобайтовые низкоуровневые операции ввода-вывода

<code>is.get(sink, size, delim)</code>	Читает до <code>size</code> байтов из потока <code>is</code> и сохраняет их в символьном массиве, начиная с адреса, на который указывает <code>sink</code> . Чтение продолжается, пока не встретится символ <code>delim</code> , либо пока не прочитано <code>size</code> байтов, либо пока не кончится файл. Если параметр <code>delim</code> присутствует, то его значение остается во входном потоке и не читается в <code>sink</code>
<code>is.getline(sink, size, delim)</code>	То же поведение, что и версии функции <code>get()</code> с тремя аргументами, но читает и отбрасывает <code>delim</code>
<code>is.read(sink, size)</code>	Читает до <code>size</code> байтов в символьный массив <code>sink</code> . Возвращает поток <code>is</code>
<code>is.gcount()</code>	Возвращает количество байтов, прочитанных из потока <code>is</code> при последним вызове функции не форматированного чтения
<code>os.write(source, size)</code>	Записывает <code>size</code> байтов из символьного массива <code>source</code> в поток <code>os</code>
<code>is.ignore(size, delim)</code>	Читает и игнорирует до <code>size</code> символов, включая <code>delim</code> . В отличие от других не форматированных функций, <code>ignore()</code> имеет аргументы по умолчанию: для <code>size</code> — 1 и для <code>delim</code> — конец файла

Функции `get()` и `getline()` имеют схожие, но не идентичные параметры. В каждом случае `sink` — это символьный массив, в который помещаются данные. Обе функции читают, пока не будет выполнено одно из следующих условий:

- Прочитано `size - 1` символов.
- Встретился конец файла.
- Встретился символ разделения.

Эти функции отличаются обработкой разделителя: функция `get()` оставляет разделитель как следующий символ потока `istream`, а функция `getline()` читает и отбрасывает разделитель. В любом случае разделитель *не сохраняется* в массиве `sink`.

**ВНИМАНИЕ**

Весьма распространенная ошибка: намереваться удалить разделитель из потока, но забыть сделать это.

Определение количества читаемых символов

Некоторые из операций читают из ввода неизвестное количество байтов. Для определения количества символов, прочитанных последней операцией не форматированного ввода, можно вызвать функцию `gcount()`. Имеет смысл вызывать функцию `gcount()` перед любым вмешательством в операции не форматированного ввода. В частности, операции с единичными символами, возвращающими их в поток, также являются операциями не форматированного ввода. Если функции `peek()`, `unget()` или `putback()` будут вызваны перед вызовом функции `gcount()`, то будет возвращено значение 0.

Упражнения раздела 17.5.2

Упражнение 17.37. Используйте не форматированную версию функции `getline()` для чтения файла по строке за раз. Проверьте программу на примере файла с пустыми строками, а также со строками, длина которых больше символьного массива, переданного функции `getline()`.

Упражнение 17.38. Дополните программу из предыдущего упражнения так, чтобы выводить каждое прочитанное слово в отдельной строке.

17.5.3. Произвольный доступ к потоку

Некоторые из потоковых классов обеспечивают произвольный доступ к данным связанного с ними потока. Положение в потоке можно изменить так, чтобы прочитать сначала последнюю строку, затем первую и т.д. Для *установки* (`seek`) необходимой позиции и *сообщения* (`tell`) текущей позиции в потоке библиотека предоставляет пару функций.



Произвольный доступ для чтения и записи напрямую зависит от системы. Чтобы выяснить способ применения этой возможности, следует обратиться к документации на систему.

Хотя функции `seek()` и `tell()` определены для всех потоковых классов, возможные для них действия определяются видом объекта, с которым связан поток. В большинстве систем поток, с которым связан потоковый объект `cin`, `cout`, `cerr` или `clog`, не обеспечивает возможности произвольного доступа — в конце концов, как можно перейти на десять позиций обратно, если запись осуществляется непосредственно в объект `cout`? Применить функции `seek()` и `tell()`, конечно, можно, но во время выполнения это приведет к ошибке и переходу потока в недопустимое состояние.



Поскольку классы `istream` и `ostream` обычно не обеспечивают произвольного доступа, в остальной части этого раздела речь идет только о классах `fstream` и `sstream`.

Функции установки и сообщения

Для обеспечения произвольного доступа типы ввода-вывода обладают маркером (marker), который указывает позицию следующей операции чтения или записи. Они обладают также двумя функциями: одна устанавливает (`seek`) маркер в новую позицию, а вторая сообщает (`tell`) текущую позицию маркера. Фактически в библиотеке определены две пары функций установки и сообщения, которые описаны в табл. 17.21. Одна пара функций используется потоками ввода, а вторая — потоками вывода. Версии для ввода и вывода различаются суффиксом. Суффикс `g` (getting) означает получение данных (чтение), а суффикс `p` (putting) — помещение данных (запись).

Таблица 17.21. Функции установки и сообщения

<code>tellg()</code> <code>tellp()</code>	Возвращает текущую позицию маркера потока ввода (<code>tellg()</code>) или поток вывода (<code>tellp()</code>)
<code>seekg(pos)</code> <code>seekp(pos)</code>	Переустанавливает маркер потока ввода или вывода на заданный параметром <code>pos</code> абсолютный адрес в потоке. Значение <code>pos</code> обычно возвращается предыдущим вызовом в соответствующей функции <code>tellg()</code> или <code>tellp()</code>
<code>seekp(off, from)</code> <code>seekg(off, from)</code>	Переустанавливает маркер потока ввода или вывода на <code>off</code> символов вперед или назад от значения <code>from</code> , которое может быть: <code>beg</code> — от начала потока; <code>cur</code> — от текущей позиции потока; <code>end</code> — от конца потока

Вполне логично, что для класса `istream`, а также производных от него классов `ifstream` и `istringstream` (см. раздел 8.1, стр. 401) можно использовать только версии `g`, а для классов `ostream` и классов `ofstream` и `ostringstream`, производных от него, можно использовать только версии `p`. Классы `iostream`, `fstream` и `stringstream` способны читать и записывать данные в поток, поэтому для них можно использовать обе версии, `g` и `p`.

Существует только один маркер

Тот факт, что библиотека различает версии функций `seek()` и `tell()` для чтения и записи, может ввести в заблуждение. Хотя библиотека и различает эти функции, в файле существует только один маркер, т.е. *нет* разных маркеров для чтения и записи.

Когда речь идет о потоке только ввода или вывода, различие не столь очевидно. В таких потоках можно использовать версии только `g` или `p`. Если попытаться вызвать функцию `tellp()` для объекта класса `ifstream`, компилятор сообщит об ошибке. Аналогично он поступит при попытке вызвать функцию `seekg()` для объекта класса `ostringstream`.

Типы `fstream` и `stringstream` допускают чтение и запись в тот же поток. У них есть один буфер для хранения подлежащих чтению и записи данных, а также один маркер, обозначающий текущую позицию в буфере. Библиотечные функции версий `g` и `p` используют тот же маркер позиции.



Поскольку существует только один маркер, для переустановки маркера при каждом переключении между чтением и записью *следует* применять функцию `seek()`.

Перемещение маркера

Имеются две версии функции установки позиции: одна обеспечивает переход к указанной позиции в файле, а другая осуществляет смещение от текущей позиции.

```
// установка маркера в заданную позицию
seekg(new_position); // установить маркер чтения в позицию pos_type
seekp(new_position); // установить маркер записи в позицию pos_type

// смещение позиции на указанную дистанцию от текущей
seekg(offset, from); // установить дистанцию смещения маркера чтения
seekp(offset, from); // от from; offset имеет тип off_type
```

Возможные значения параметра `from` перечислены в табл. 17.21 (стр. 957).

Аргументы `new_position` и `offset` этих функций имеют машинно-зависимые типы `pos_type` и `off_type` соответственно. Они определены в классах `istream` и `ostream`. Тип `pos_type` представляет позицию файла, а тип `off_type` — смещение от этой позиции. Значение типа `off_type` может

быть положительным или отрицательным, что соответствует смещению вперед или назад.

Доступ к маркеру

Функции `tellg()` и `tellp()` возвращают значение типа `pos_type`, обозначающее текущую позицию в потоке. Эти функции обычно используются для того, чтобы запомнить позицию и впоследствии вернуться к ней:

```
// запомнить текущую позицию записи в переменную mark
ostream writeStr; // поток вывода в строку
ostream::pos_type mark = writeStr.tellp();
// ...
if (cancelEntry)
    // возврат к отмеченной позиции
    writeStr.seekp(mark);
```

Чтение и запись в тот же файл

Рассмотрим пример программы, которая читает файл и записывает в его конец новую строку, содержащую относительную позицию начала каждой строки. Предположим, например, что работать придется со следующим файлом.

```
Abcd
efg
hi
j
```

Модифицированный программой файл должен выглядеть следующим образом.

```
Abcd
efg
hi
j
5 9 12 14
```

Обратите внимание: программа не записывает смещение для первой строки, она всегда начинается с позиции 0. Обратите также внимание на то, что смещения должны также учитывать невидимый символ новой строки, завершающий каждую строку. И наконец, последнее число в выводе — смещение для строки, с которой начинается вывод. При включении этого смещения в вывод можно отличить свой вывод от первоначального содержимого файла. Можно прочесть последнее число в полученном файле и установить смещение так, чтобы получить позицию начала вывода.

Наша программа будет читать файл построчно. Для каждой строки значение счетчика будет увеличиваться на размер только что прочитанной строки. Этот счетчик содержит смещение, с которого начинается следующая строка:

```
int main()
{
```

```

// открыть файл для ввода и вывода, а затем перейти в его конец
// аргументы режима файла приведены в табл. 8.4 (стр. 411)
fstream inOut("copyOut",
              fstream::ate | fstream::in | fstream::out);
if (!inOut) {
    cerr << "Unable to open file!" << endl;
    return EXIT_FAILURE; // EXIT_FAILURE см. р. 6.3.2 (стр. 298)
}
// inOut открыт в режиме ate, поэтому исходной позицией файла будет
// его конец
auto end_mark = inOut.tellg(); // запомнить позицию первоначального
                               // конца файла
inOut.seekg(0, fstream::beg); // перейти к началу файла
size_t cnt = 0;              // счетчик количества байтов
string line;                  // содержит каждую строку ввода
// пока нет ошибки и исходные данные читаются
while (inOut && inOut.tellg() != end_mark
      && getline(inOut, line)) { // и можно получить следующую строку
    cnt += line.size() + 1;      // добавить 1 для новой строки
    auto mark = inOut.tellg();   // запомнить позицию чтения
    inOut.seekp(0, fstream::end); // установить маркер записи в конец
    inOut << cnt;                // записать общую длину
    // вывести разделитель, если это не последняя строка
    if (mark != end_mark) inOut << " ";
    inOut.seekg(mark); // восстановить позицию чтения
}
inOut.seekp(0, fstream::end); // перейти к концу
inOut << "\n";                // вывести символ новой строки в конце файла
return 0;
}

```

Эта программа открывает поток `fstream` в режимах `in`, `out` и `ate` (см. табл. 8.4, стр. 411). Первые два режима означают, что предполагается чтение и запись в тот же файл. Режим `ate` означает, что начальной позицией открытого файла будет его конец. Как обычно, необходимо удостовериться, что файл открыт корректно, если это не так, следует выйти из программы (см. раздел 6.3.2, стр. 298).

Поскольку программа пишет в свой исходный файл, нельзя использовать конец файла как признак прекращения чтения. Цикл должен закончиться по достижении конца первоначального ввода. В результате сначала следует запомнить первоначальную позицию конца файла. Так как файл открыт в режиме `ate`, поток `inOut` уже установлен в конец. Сохраним текущую (т.е. первоначальную) позицию конца файла в переменной `end_mark`. Запомнив конечную позицию, маркер чтения следует установить в начало файла, чтобы можно было приступить к чтению данных.

Цикл `while` имеет три условия выхода: сначала проверяется допустимость потока; если это так, то проверяется, не достигнут ли конец исходных данных. Для этого текущая позиция чтения, возвращаемая функцией `tellg()`, сравнивается с позицией, заранее сохраненной в переменной `end_mark`. И нако-

нец, если обе проверки пройдены успешно, происходит вызов функции `getline()`, которая читает следующую строку из файла. Если вызов функции `getline()` успешен, выполняется тело цикла.

Тело цикла начинается с запоминания текущей позиции в переменной `mark`. Она сохраняется для возвращения после записи следующего относительного смещения. Вызов функции `seekp()` переводит маркер записи в конец файла. Выводится значение счетчика, а затем функция `seekg()` возвращается к позиции, сохраненной в переменной `mark`. Восстановив положение маркера, можно снова проверить условие выхода из цикла `while`.

Каждая итерация цикла выводит смещение следующей строки. Поэтому последняя итерация цикла заботится о записи смещения последней строки. Однако в конец файла следует еще записать символ новой строки. Как и в других случаях записи, для позиционирования в конец файла перед выводом новой строки происходит вызов функции `seekp()`.

Упражнения раздела 17.5.3

Упражнение 17.39. Напишите собственную версию программы, представленной в этом разделе.

Резюме

В этой главе рассматривались дополнительные операции ввода-вывода и четыре библиотечных типа: кортеж, набор битов, регулярные выражения и случайные числа.

Шаблон `tuple` (кортеж) позволяет объединять члены несоизмеримых типов в единый объект. Каждый кортеж содержит конкретное количество членов, но библиотека не налагает ограничений на их количество.

Тип `bitset` (набор битов) позволяет определять коллекции битов определенного размера. Размер набора битов не ограничен размером любого из целочисленных типов и вполне может превышать их. Кроме поддержки обычных побитовых операторов (см. раздел 4.8, стр. 210), набор битов определяет несколько специальных операторов, которые позволяют манипулировать состоянием отдельных битов в наборе.

Библиотека регулярных выражений предоставляет коллекцию классов и функций: класс `regex` представляет регулярные выражения, написанные на одном из нескольких общепринятых языков регулярных выражений. Классы соответствия содержат информацию о конкретном соответствии. Они используются функциями `regex_search()` и `regex_match()`. Эти функции получают объект класса `regex` и последовательность символов, а затем обнаруживают соответствия регулярного выражения `regex` в данной последовательно-

сти символов. Итераторы типа `regex` являются адаптерами итераторов, используемых функцией `regex_search()` для перебора исходной последовательности и возвращения каждого соответствия. Есть также функция `regex_replace()`, позволяющая заменять соответствующие части заданной исходной последовательности указанной альтернативой.

Библиотека случайных чисел — это коллекция процессоров случайных чисел и классов распределения. Процессор случайных чисел возвращает последовательность равномерно распределенных целочисленных значений. Библиотека определяет несколько процессоров с разной производительностью. Процессор `default_random_engine` определен как подходящий для большинства случаев. Библиотека определяет также 20 типов распределений. Эти типы распределений используют процессор как источник случайных чисел определенного типа в заданном диапазоне, которые распределены согласно заданной вероятности распределения.

Термины

Генератор случайных чисел (`random-number generator`). Комбинация типа процессора случайных чисел и типа распределения.

Исключение `regex_error`. Тип исключения, передаваемого при синтаксической ошибке в регулярном выражении.

Итератор `cregex_iterator`. Похож итератору `sregex_iterator`, но перебирает массив типа `char`.

Итератор `sregex_iterator`. Итератор, перебирающий строку с использованием заданного объекта класса `regex` для поиска соответствий в заданной строке. При вызове функции `regex_search()` конструктор позиционирует итератор на первое соответствие. Приращение итератора вызывает функцию `regex_search()`, начиная сразу после текущего соответствия в данной строке. Обращение к значению итератора возвращает объект класса `smatch`, описывающий текущее соответствие.

Класс `bitset` (набор битов). Определенный в стандартной библиотеке класс, объект которого содержит коллекцию битов, размер которой известен на момент компиляции, и позволяет выполнять с ним операции по проверке и установке значений.

Класс `smatch`. Контейнер объектов типа `csub_match`, предоставляющий информацию о соответствии классу `regex` в исходной последовательности типа `const char*`. Первый элемент в контейнере описывает общие результаты поиска соответствия. Последующие элементы описывают результаты для подвыражений.

Класс `regex`. Класс, обслуживающий регулярное выражение.

Класс `smatch`. Контейнер объектов типа `csub_match`, предоставляющий информацию о соответствии классу `regex` в исходной последовательности типа `string`. Первый элемент в контейнере описывает общие результаты поиска соответствия. Последующие элементы описывают результаты для подвыражений.

Манипулятор (manipulator). Подобный функции объект, “манипулирующий” потоком. Манипуляторы применяются как правый операнд на перегруженные операторы ввода-вывода, << и >>. Большинство манипуляторов изменяет внутреннее состояние объекта. Они зачастую предоставляются парами: один изменяет состояние потока, а второй возвращает поток в стандартное состояние.

Младшие биты (low-order). Биты набора, обладающие самыми маленькими индексами.

Начальное число (seed). Значение, предоставляемое процессору случайных чисел, чтобы перейти к новому пункту в последовательности создаваемых чисел.

Не форматированный ввод-вывод (unformatted IO). Операции, рассматривающие поток как недифференцированный поток байтов. Не форматированные операции налагают все обязанности по управлению вводом и выводом на пользователя.

Подвыражение (subexpression). Заключенный в скобки компонент схемы регулярного выражения.

Процессор случайных чисел (random-number engine). Библиотечный тип, позволяющий создавать беззнаковые случайные числа. Процессоры предназначены для использования только как источники для распределения случайных чисел.

Распределение случайных чисел (random-number distribution). Тип стандартной библиотеки, преобразующий вывод процессора случайного числа согласно его именованному распределению. Например, шаблон `uniform_int_distribution<T>` создает однородно распределенные целые числа типа `T`, шаблон `normal_distribution<T>` создает числа с нормальным распределением и т.д.

Регулярное выражение (regular expression). Способ описания последовательности символов.

Стандартный процессор случайных чисел (default random engine). Псевдоним типа для процессора случайных чисел, предназначенный для обычного использования.

Старшие биты (high-order). Биты набора, обладающие самыми большими индексами.

Тип `csub_match`. Тип, содержащий результаты поиска соответствия регулярного выражения для типа `const char*`. Может представлять все соответствия или подвыражение.

Тип `ssub_match`. Тип, содержащий результаты поиска соответствия регулярного выражения для типа `string`. Может представлять все соответствия или подвыражение.

Форматированный ввод-вывод (formatted IO). Операции ввода-вывода, использующие для определения действий операций типы читаемых или записываемых объектов. Поскольку сложные операции ввода выполняют все соответствующие читаемому типу преобразования, такие как преобразование числовых строк ASCII в указанный арифметический тип, отступ (по умолчанию) игнорируется. Процедуры форматированного вывода преобразуют типы в представления отображаемых символов, дополняя (возможно) вывод и выполняя другие, специфические для типа преобразования.

Функция `regex_match()`. Функция, сообщающая, соответствует ли вся исходная последовательность заданному объекту класса `regex`.

Функция `regex_replace()`. Функция, использующая объект класса `regex` для замены соответствующего подвыражения исходной последовательности с использованием заданного формата.

Функция `regex_search()`. Функция, использующая объект класса `regex` для поиска последовательности соответствия в заданной исходной последовательности.

Шаблон `tuple` (кортеж). Шаблон, позволяющий создавать типы для хранения безымянных членов определенных типов. Нет никаких ограничений на количество членов, для содержания которых может быть определен кортеж.

Шаблон функции `get`. Шаблон функции, возвращающий определенный элемент для заданного кортежа. Например, функция `get<0>(t)`, возвращает первый элемент из кортежа `tuple t`